



UNIVERSITY OF MACEDONIA
MSc IN ENTERPRISE SOFTWARE SYSTEMS DEVELOPMENT
DEPARTMENT OF APPLIED INFORMATICS

**DEVELOPMENT AND EVALUATION OF AUTOMATED
CI/CD PIPELINE FOR KUBERNETES-BASED
MICROSERVICES**

MSc Dissertation

by

Alexios Stamelos

Thessaloniki, Month 2026

DEVELOPMENT AND EVALUATION OF AUTOMATED CI/CD PIPELINE FOR KUBERNETES-BASED MICROSERVICES

Alexios Stamelos

BSc Marine Science

MSc Quality Management and Production Organization Systems for the Food Industry

MSc Dissertation

submitted in partial fulfillment of the requirements for the

MSc IN ENTERPRISE SOFTWARE SYSTEMS DEVELOPMENT

Declaration of Originality

I declare that this dissertation is entirely my own original work and has been composed by me for the award of this degree. It has not been submitted, in whole or in part, for any other degree or qualification. All sources, ideas, words, data, tables, figures, and other material derived from the work of others have been properly acknowledged and referenced. I further declare that this dissertation does not contain plagiarism, fabrication, falsification, ghostwriting, or any other form of academic misconduct. No generative artificial intelligence tool has been used to produce substantive parts of the text, arguments, analysis, or conclusions of this dissertation without explicit permission and disclosure in accordance with the University's regulations. Any permitted digital tools used for minor language, spelling, grammar, or formatting assistance did not replace my own intellectual contribution, and I take full responsibility for the content, accuracy, and integrity of this work.

Supervisor

Panagiotis Papadimitriou

Approved by the three-member examination committee on dd/mm/yyyy

Panagiotis Papadimitriou

Kaskalis Theodoros

Koloniari Georgia

.....

.....

.....

Alexios Stamelos

Abstract

The transition from monolithic software architectures to distributed microservices has introduced significant deployment complexities, necessitating orchestration platforms such as Kubernetes and robust automation through DevOps methodologies. While traditional "push-based" Continuous Integration and Continuous Delivery (CI/CD) pipelines address some of these challenges, they frequently suffer from security vulnerabilities, delayed defect discovery, and pipeline brittleness. This thesis presents the design, implementation, and empirical evaluation of a declarative, "pull-based" GitOps CI/CD architecture specifically tailored for Kubernetes-based microservices. By decoupling the integration and delivery phases utilizing Jenkins and Argo CD across a strict multi-repository topology, the proposed architecture successfully abstracts administrative cluster access away from the CI server, thereby enhancing the overall security posture. A critical contribution of this study is the implementation of a "Shift-Left" testing paradigm. By programmatically synthesizing ephemeral, localized container orchestration environments and executing End-to-End (E2E) automation via Playwright, the pipeline establishes a definitive quality gate. This mechanism proactively intercepts both user interface regressions and catastrophic inter-service dependency failures, physically insulating the production cluster from defective logic. Furthermore, the architecture resolves inherent CI infrastructure vulnerabilities by incorporating defensive programming algorithms to handle structural codebase mutations during massive upstream synchronizations, alongside declarative "janitor" routines that automate aggressive resource hygiene. The system was empirically validated through five distinct operational scenarios. The results demonstrate successful continuous delivery, zero-touch disaster recovery via autonomous reconciliation of state drift, and high-fidelity runtime validation. Ultimately, this research proves that combining GitOps principles with ephemeral testing environments provides a highly resilient, scalable, and auditable operational baseline capable of supporting continuous software evolution.

Keywords: GitOps, Git, Jenkins, ArgoCD, Playwright, Oracle VirtualBox, Kubernetes, Helm, Continuous Integration, Continuous Delivery.

Περίληψη

Η μετάβαση σε αρχιτεκτονικές με microservices αυξάνει δραματικά την πολυπλοκότητα της ανάπτυξης λογισμικού (deployments). Οι παραδοσιακές "push-based" προσεγγίσεις παρουσιάζουν σημαντικούς περιορισμούς ως προς την ασφάλεια και την αξιοπιστία. Για την αντιμετώπιση αυτών των προκλήσεων, στην παρούσα διπλωματική εργασία αναπτύχθηκε και αξιολογήθηκε εμπειρικά μια προηγμένη αρχιτεκτονική Continuous Integration / Continuous Delivery (CI/CD) βασισμένη στο πρότυπο GitOps.

Το σύστημα αναπτύχθηκε τοπικά μέσω του Oracle VirtualBox, αξιοποιώντας τέσσερις εικονικούς υπολογιστές (VMs): έναν Kubernetes Master, δύο Kubernetes Workers και έναν διακομιστή αυτοματισμού Jenkins. Ως εφαρμογή ελέγχου χρησιμοποιήθηκε το Google Online Boutique (GOB). Το Continuous Delivery Pipeline (CDP) υλοποιήθηκε μέσω του ArgoCD, το οποίο ανέλαβε το deployment της εφαρμογής στο τοπικό cluster αντλώντας δεδομένα από ένα forked repository.

Το Continuous Integration Pipeline (CIP) αναπτύχθηκε σε κώδικα Groovy (Jenkins code) και εκτελείται σε έξι διαδοχικά στάδια. Αρχικά (Checkout & Smart Detect), το σύστημα εντοπίζει δυναμικά μέσω git diff τις τροποποιημένες υπηρεσίες. Στη συνέχεια (Build Images), μεταγλωττίζει τοπικά τα Docker images και τα κάνει tag. Ακολούθως (Spin Up Ephemeral Environment), δημιουργεί ένα πλήρως λειτουργικό, εφήμερο (Ephemeral) αντίγραφο της εφαρμογής στον CI server μέσω docker-compose. Σε αυτό το περιβάλλον εκτελούνται Playwright End-to-End δοκιμές (E2E Testing), λειτουργώντας ως η απόλυτη πύλη ποιότητας (Quality Gate). Αν οι δοκιμές επιτύχουν, τα images προωθούνται στο registry (Push to Docker Hub) και το pipeline ενημερώνει το αρχείο values.yaml του Helm (Update GitOps Repo), ενεργοποιώντας τον συγχρονισμό (synchronization) από το ArgoCD. Κάθε εκτέλεση ολοκληρώνεται με ένα Post στάδιο ("Janitor" block) για τη διαγραφή των containers και τον καθαρισμό των πόρων, αποτρέποντας την εξάντληση της μνήμης.

Η αρχιτεκτονική αξιολογήθηκε μέσω πέντε ρεαλιστικών σεναρίων: Στο πρώτο σενάριο (Safe Feature Enhancement) επιβεβαιώθηκε το αυτοματοποιημένο deployment χωρίς downtime. Στο δεύτερο (UI Regression Detection - Shift-Left Quality Gate), το σύστημα εντόπισε επιτυχώς προσομοιωμένα σφάλματα διεπαφής, προστατεύοντας το cluster παραγωγής. Στο τρίτο (Inter-Service Dependency Failure) αποδείχθηκε ότι τα εφήμερα περιβάλλοντα μπορούν να εντοπίσουν

βαθύτερα σφάλματα εξαρτήσεων που διαφεύγουν από τις απλές δοκιμές. Στο τέταρτο (Unauthorized State Drift), το ArgoCD αποκατέστησε αυτόνομα τη μη εξουσιοδοτημένη διαγραφή πόρων (self-healing). Τέλος, στο πέμπτο σενάριο (Upstream Synchronization and Bulk Integration) αντιμετωπίστηκε αποτελεσματικά η κατασκευαστική ευθραυστότητα (pipeline brittleness) κατά τη μαζική ενσωμάτωση αλλαγών.

Εν κατακλείδι, η σύζευξη των αρχών του GitOps με τα εφήμερα περιβάλλοντα δοκιμών δημιουργεί ένα εξαιρετικά ανθεκτικό, ασφαλές και ελεγχόμενο πλαίσιο λειτουργίας, ικανό να υποστηρίξει τη συνεχή εξέλιξη σύγχρονου λογισμικού μεγάλης κλίμακας.

Λέξεις Κλειδιά: GitOps, Git, Jenkins, ArgoCD, Playwright, Oracle VirtualBox, Kubernetes, Helm, Continuous Integration, Continuous Delivery.

Contents

1. Introduction.....	13
1.1 Motivation.....	13
1.2 Contribution	14
1.3 Structure.....	16
2. Literature Review.....	17
2.1 Evolution of Micro-service Architectures.....	17
2.2 Containerization and Container Orchestration.....	19
2.2.1 Kubernetes	19
2.2.2 Helm.....	23
2.3 DevOps Methodologies and CI/CD Principles	25
2.4 CI/CD Approaches Specific to Kubernetes.....	28
3. Methodology.....	31
3.1 Infrastructure & Deployment.....	31
3.2 Repository Architecture and Connectivity.....	32
3.3 Baseline Deployment	33
3.4 Automated CI/CD Pipeline and Shift-Left Testing Analysis	34
3.4.1 Intelligent Change Detection and Local Build Phase	34
3.4.2 Ephemeral Environment Instantiation and E2E Testing	36
3.4.3 Quality Gate and Defective Code Mitigation	38
3.4.4 Continuous Delivery via Argo CD (GitOps)	38
4. Experimental Study.....	40
4.1 Scenario 1: Safe Feature Enhancement (Continuous Delivery Validation)	40
4.2 Scenario 2: UI Regression Detection (Shift-Left Quality Gate).....	44
4.3 Scenario 3: Inter-Service Dependency Failure (Catastrophic Crash Prevention).....	46
4.4 Scenario 4: Unauthorized State Drift (GitOps Self-Healing)	47
4.5 Scenario 5: Upstream Synchronization and Bulk Integration.....	49
5. Conclusions.....	53
5.1 Evaluation of the GitOps Paradigm	53
5.2 Shift-Left Testing and Inter-Service Validation	53

5.3 Ephemeral Orchestration and Test Idempotency	54
5.4 Autonomous Reconciliation and Disaster Recovery.....	55
5.5 Pipeline Brittleness and Infrastructural Hygiene	55
5.6 Security Posture and Multi-Repository Topology	56
5.7 Technical Limitations and Future Directions.....	56
Appendix.....	58
Repositories and uploaded files	58
Other Figures	58
Bibliography	64

List of Figures

Figure 1: Structure of a template microservice for the parcel service web portal	18
Figure 2: Architectural evolution of an application	19
Figure 3: Kubernetes architecture-cluster	20
Figure 4: Software development pipeline diagram	26
Figure 5: Kubernetes lab infrastructure.	31
Figure 6: Cluster network configuration	32
Figure 7: Baseline network topology	34
Figure 8: Architectural flow of the GitOps-driven CI/CD pipeline	36
Figure 9: Current state of the website before the changes.	40
Figure 10: Changes shown in the GitHub before merge	41
Figure 11: Jenkins detection	41
Figure 12: Out of sync state of the app.	43
Figure 13: Synced state of the app.	44
Figure 14: Deployed application.....	44
Figure 15: Code changes	45
Figure 16: Sync state of the application.....	46
Figure 17: Terminal interface.....	48
Figure 18: Argo CD continuous delivery dashboard detecting the environmental drift.	49
Figure 19: Jenkins detection	50
Figure 20: Kubernetes Pod Snapshot.....	59
Figure 21: Kubernetes dashboard	60
Figure 22: Argo CD snapshot	61
Figure 23: Terminal output of the kubectl cluster-info command	61
Figure 24: Terminal output showing the detailed status of the Kubernetes cluster nodes.....	61
Figure 25: Terminal output of the kubectl get services command.	62
Figure 26: Oracle VirtualBox Manager Snapshot.....	62
Figure 27: Terminal output of the kubectl top nodes command	63

1. Introduction

1.1 Motivation

In the contemporary landscape of enterprise software engineering, the transition from traditional monolithic applications to distributed microservice architectures has become a foundational paradigm shift (Dragoni et al., 2017). While monoliths suffer from technology lock-in and high-risk deployments where a single code change can necessitate a complete system reboot, microservices functionally decompose applications into independent, loosely coupled processes. This architectural style grants development teams the autonomy to maintain, test, and deploy individual services with minimal downtime. However, this distribution introduces immense deployment complexity, as architecture now relies on an intricate web of network communications and isolated release cycles (Dragoni et al., 2017, Newman, 2015). To manage this growing complexity, containerization and orchestration platforms, most notably Kubernetes, have emerged as absolute necessities, acting as the operating system for clusters to handle service discovery, load balancing, and dynamic scaling (Burns et al., 2016, Dragoni et al., 2017, Koukis et al., 2024).

Despite the infrastructural advantages provided by Kubernetes, managing deployments manually within such complex ecosystems remains highly error-prone and inefficient (Bass et al., 2015, Forsgren et al., 2018). The DevOps cultural movement emerged to address these challenges by advocating for high degrees of automation, standardization, and continuous delivery to reduce deployment lead times without compromising product quality (Forsgren et al., 2018, Humble and Farley, 2010, Kim et al., 2016). Traditional approaches to Continuous Integration and Continuous Delivery (CI/CD) have historically relied on "push-based" deployment models, where automation servers directly execute deployment scripts against target environments (Bass et al., 2015, Paavola, 2021). However, these centralized push models present significant security and reliability concerns, as the CI server requires direct administrative access to the production cluster, leaving it vulnerable to manual configuration drift. This operational risk strongly motivates the transition toward the "pull-based" GitOps paradigm, which utilizes tools like Argo CD to continuously monitor a version-controlled repository as the absolute single source of truth and autonomously synchronize the cluster state (Paavola, 2021).

Another critical motivation for this study stems from the inherent difficulties of integration testing in distributed systems (Dragoni et al., 2017, Newman, 2015). Historically, user interface

regressions and deep inter-service dependency failures were frequently discovered far too late in the deployment lifecycle, often after defective code had already breached staging or production environments (Duvall et al., 2007, Humble and Farley, 2010). Traditional continuous integration practices primarily focus on syntax verification and artifact compilation, which are fundamentally insufficient for validating dynamic runtime behaviors or complex backend communications (Humble and Farley, 2010). Furthermore, maintaining permanent staging clusters for full-stack integration testing incurs substantial cloud computing costs and introduces severe vulnerabilities related to state pollution when multiple developers run concurrent tests (Bass et al., 2015, Newman, 2015). Consequently, there is a pressing need to implement a "Shift-Left" testing philosophy that utilizes localized, ephemeral environments to rigorously validate code prior to deployment, bridging the gap between developer workstations and live clusters without the financial overhead of persistent infrastructure (Bass et al., 2015, Forsgren et al., 2018).

Finally, the operational realities of maintaining active forks of open-source repositories highlight severe vulnerabilities within the continuous integration pipelines themselves (Bass et al., 2015). In modern cloud-native development, organizations frequently rely on distributed teams committing bulk updates, which can expose "pipeline brittleness"—a phenomenon where rigid automation scripts catastrophically fail due to unanticipated structural mutations in upstream repositories (Bass et al., 2015, Humble and Farley, 2010). Furthermore, the intense computational and storage overhead required to dynamically build, tag, and orchestrate dozens of microservice containers locally can easily culminate in fatal resource exhaustion events, such as disk space depletion on the runner node (Bass et al., 2015, Newman, 2015). These real-world infrastructural challenges motivate the engineering of resilient, defensive pipeline logic and aggressive automated hygiene mechanisms to guarantee long-term stability in localized CI/CD environments (Bass et al., 2015).

1.2 Contribution

The primary contribution of this applied engineering study is the design, implementation, and empirical evaluation of a declarative GitOps CI/CD pipeline specifically tailored for Kubernetes-based microservices. By successfully decoupling the continuous integration phase from the continuous delivery phase, this architecture mitigates the operational risks associated with traditional deployment models. The implementation establishes a strict multi-repository topology

that securely segregates the application source code, the declarative GitOps configurations, and the automated testing frameworks. This separation of concerns ensures that the continuous integration server is granted only the precise permissions required for integration validations, while the continuous delivery controller retains exclusive responsibility for interpreting the validated configurations and deploying them to the live cluster.

A significant technical contribution is the development of a highly optimized, ephemeral container orchestration environment utilized for runtime validation. By programmatically synthesizing the microservice topology directly upon the CI execution node, the architecture provisions an unadulterated, transient replica of the production environment for every discrete codebase commit. Coupled with a robust End-to-End testing suite, this mechanism serves as a definitive quality gate. The study empirically demonstrates that this localized "Shift-Left" testing strategy proactively intercepts both user interface regressions and catastrophic inter-service dependency failures, physically insulating the production cluster from defective logic and providing an immediate feedback loop for development teams.

Furthermore, the research practically validates the autonomous self-healing capabilities and disaster recovery advantages inherent to the declarative GitOps philosophy. Through controlled experimental scenarios involving malicious or erroneous infrastructural tampering, the study proves that an active reconciliation loop can instantaneously recognize deviations between the live cluster state and the version-controlled configuration. The pipeline successfully demonstrates zero-touch recovery capabilities, autonomously regenerating missing infrastructure without the need for external webhook triggers or human intervention, thereby fundamentally reducing the mean time to recovery for localized degradation.

Lastly, this thesis contributes directly to the engineering of robust automation infrastructure by actively addressing pipeline brittleness and resource exhaustion. The study documents the refactoring of integration logic to incorporate defensive programming principles, specifically through dynamic file-existence validation algorithms capable of absorbing massive, unpredictable architectural mutations during upstream bulk-merges. Additionally, the architecture implements a mandatory declarative post-execution cleanup routine—a "janitor" block—that systematically executes aggressive container teardowns and forced image pruning. This automated hygiene mechanism successfully reclaims critical disk space, proving that aggressive resource management

is a mandatory requirement to prevent CI automation tools from inadvertently degrading their own hosting infrastructure.

1.3 Structure

The remainder of this thesis is structured to provide a logical progression from the foundational theories to the practical implementation, experimental evaluation, and critical analysis of the developed CI/CD architecture. Chapter 2 establishes the theoretical background by reviewing the literature surrounding modern software development paradigms. It explores the evolution from monolithic systems to microservices, the necessity of containerization and Kubernetes orchestration, and the core principles of DevOps, contrasting traditional continuous integration approaches with modern GitOps methodologies.

Chapter 3 details the methodology employed in this study. It outlines the provisioning of the virtualized Kubernetes lab infrastructure, the strategic design of the multi-repository architecture, and the deployment of the baseline microservices application. Furthermore, it provides an in-depth operational breakdown of the automated CI/CD pipeline, explaining the intelligent change detection mechanisms, the instantiation of the ephemeral testing environment, and the final continuous delivery handover.

Chapter 4 presents the Experimental Study, documenting the empirical results obtained from subjecting the architecture to five distinct operational scenarios. These scenarios include safe feature enhancements, the detection of UI regressions, the prevention of catastrophic backend crashes, the autonomous healing of unauthorized state drift, and the integration of bulk upstream synchronizations. Finally, Chapter 5 delivers a discussion of the findings, evaluating the efficacy of the GitOps paradigm and the Shift-Left testing strategy, while also acknowledging the technical limitations of the study and proposing promising avenues for future research and architectural enhancements.

2. Literature Review

The primary objective of this chapter is to establish the theoretical foundation for the thesis by reviewing the literature surrounding modern software architecture, delivery pipelines, and operational paradigms. The chapter begins by contrasting traditional monolithic applications with distributed microservices, highlighting the deployment challenges the latter introduces (Dragoni et al., 2017, Newman, 2015). It then explores containerization and the necessity of orchestrator platforms like Kubernetes to manage these environments (Burns et al., 2016). Following this, the chapter defines DevOps methodologies and Continuous Integration/Continuous Delivery (CI/CD) principles, transitioning into specific pipeline approaches and the metrics used to evaluate them (Humble and Farley, 2010, Forsgren et al., 2018).

2.1 Evolution of Micro-service Architectures

Historically, software development relied heavily on monolithic architectures, which represent single, indivisible applications that often grow alarmingly complex and difficult to maintain over time (Dragoni et al., 2017). Monoliths represent a technology lock-in that binds developers to the original programming languages and frameworks. Furthermore, a single-line code change in a monolithic application can force a high-risk deployment of the entire system, leading to delayed releases and accumulated changes that increase the likelihood of production errors (Humble and Farley, 2010).

In contrast, modern distributed systems have embraced the microservices architectural style, which manages growing system complexity by functionally decomposing large applications into a set of independent, loosely coupled services (Dragoni et al., 2017). In the academic and industry definitions, microservice is a cohesive, independent process that interacts via messages and focuses on providing a single business capability (Newman, 2015). The benefits of this architecture are substantial: it enables developers to maintain, test, and deploy individual services with very short downtimes without requiring a complete system reboot. It also allows for technology independence, empowering teams to select the most appropriate tools for their specific service without being constrained by the choices of other teams (Newman, 2015, Dragoni et al., 2017).

As microservices functionally decompose applications, the mechanisms by which these independent services communicate become critically important. Traditionally, inter-service

communication has relied heavily on REST (Representational State Transfer) APIs over HTTP/1.1 protocols, which utilize human-readable JSON payloads. While REST remains the de facto standard for external, client-facing interfaces due to its universal compatibility, it introduces significant serialization overhead and latency when utilized for high-volume, internal microservice-to-microservice communication. Consequently, modern distributed architectures increasingly adopt gRPC (gRPC Remote Procedure Calls) operating over HTTP/2. gRPC utilizes Protocol Buffers—a strongly typed, binary serialization format—which drastically reduces payload size and allows for highly efficient, multiplexed, and synchronous communication between backend services. Understanding the distinction between these protocols is vital, as a single user request at the ingress gateway often triggers a cascading tree of both REST and gRPC internal calls to fulfill the business logic (Arora et al., 2024, gRPC 2024).

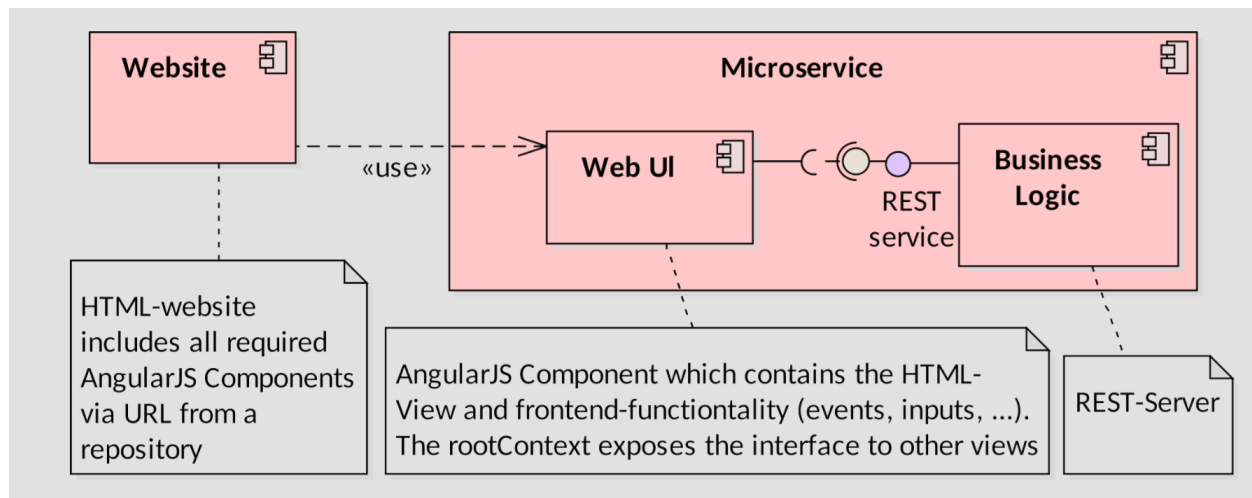


Figure 1: Structure of a template microservice for the parcel service web portal. The diagram illustrates how individual microservices encapsulate both a Web User Interface (UI) and a business logic component. The main website acts as a template, dynamically including the microservice's UI component via a URL hosted on an asset server. The business logic is exposed through a RESTful interface, which can be utilized by the UI components or other microservices within the system (Greising et al., 2018).

However, microservices introduce inherent complexities and specific deployment challenges. While monoliths suffer from tight coupling, microservices require meticulous management of network communication; chatty communication across boundaries can lead to a tangled, brittle architecture if integration is not handled carefully (Newman, 2015). Deploying microservices drastically increases the number of moving parts in a system, making deployments incredibly

complex and risky if the services lack proper isolation or if automated deployment pipelines are not established (Bass et al., 2015).

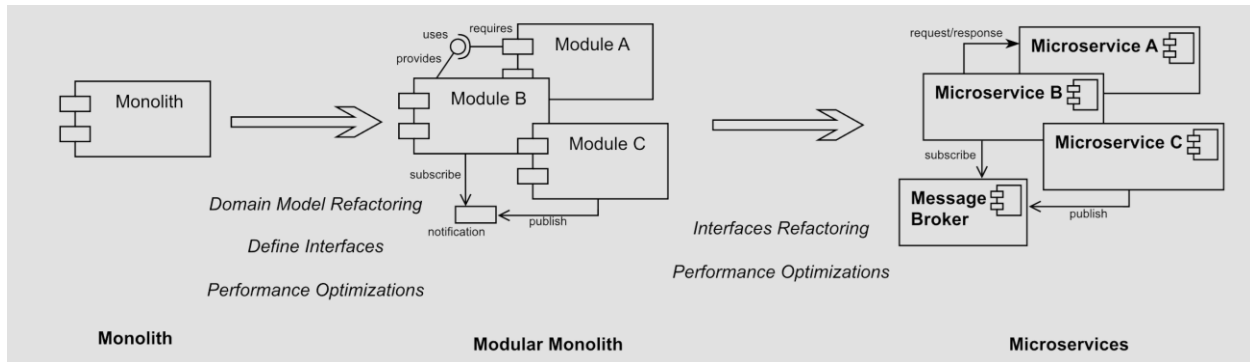


Figure 2: Architectural evolution of an application, from Monolith to Microservices. A two-phase migration illustrating the initial domain refactoring into a Modular Monolith, followed by interface refactoring to extract the modules into distributed Microservices (Faustino et al., 2024).

2.2 Containerization and Container Orchestration

To solve the deployment and environmental challenges introduced by microservices, literature points to container technologies (like Docker) (Dragoni et al., 2017). Microservices naturally lend themselves to containerization, which provides engineers with a high degree of freedom in configuring their deployment environments. Containers encapsulate the application environment, abstracting away the specific details of the host machines and operating systems. This decoupling eliminates implicit host OS dependencies and solves environment inconsistencies by ensuring that the exact same deployment environment is used across local development, testing, and production (Bass et al., 2015).

As microservice architectures scale, container orchestration becomes an absolute necessity. Without an orchestration system, engineers would be forced to manually develop and maintain complex features required for large-scale systems, such as service discovery, load balancing, and cluster management (Burns et al., 2016). Orchestration platforms handle the replication of services and the intelligent distribution of those replicas across the available nodes, enabling the system to scale dynamically based on load.

2.2.1 Kubernetes

Kubernetes has emerged as a robust, de facto standard for this orchestration (Koukis et al., 2024). Kubernetes is an open-source orchestrator for deploying, managing, and scaling containerized

applications. Originally developed by Google, it abstracts away the underlying hardware infrastructure, allowing to run software components across thousands of computer nodes as if they were a single enormous computer. By acting essentially as an operating system for a cluster, Kubernetes relieves developers from manually implementing infrastructure-related services—such as service discovery, load-balancing, and self-healing—into their applications. It provides a powerful platform for orchestrating and managing complex distributed systems across clusters. Kubernetes attempts to avert deployment complexity by adopting a highly consistent, declarative approach to its APIs (utilizing ObjectMetadata, Specification, and Status) (Burns et al., 2016). By shifting the management focus from individual machines to application instances, orchestration platforms drastically improve application deployment, scaling, and introspective monitoring.

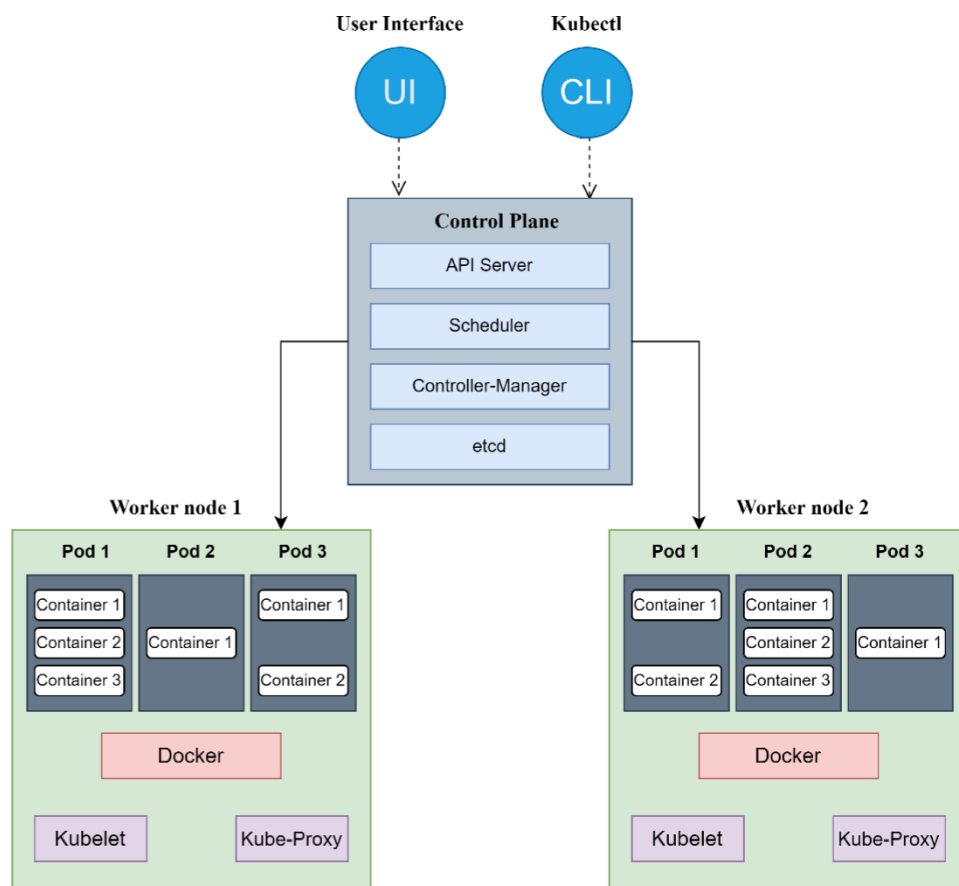


Figure 3: Kubernetes architecture-cluster (Farid et al., 2025).

A Kubernetes cluster is fundamentally divided into two parts: the Control Plane and the Worker Nodes (Luksa 2017, Burns et al., 2022).

The Control Plane constitutes the central management layer of a Kubernetes cluster and is responsible for maintaining the cluster's overall state and coordinating its operations. It typically runs on dedicated control-plane or master nodes and includes several core components, most notably the kube-apiserver, kube-scheduler, kube-controller-manager, and etcd, while some deployments may also include a cloud-controller-manager. The kube-apiserver serves as the main entry point to the cluster by exposing the Kubernetes API, through which internal components and external clients communicate with the system. The scheduler is responsible for selecting appropriate worker nodes for Pods that have not yet been assigned to a node, taking into account factors such as resource availability and scheduling constraints. The controller manager runs a set of control loops that continuously compare the actual cluster state with the desired state and take corrective actions when differences are detected. Meanwhile, etcd functions as a consistent and highly available key-value store that preserves the configuration and state data required by the cluster. For this reason, the Control Plane does not execute the actual user application containers; instead, it performs orchestration decisions, stores cluster state, and directs worker nodes to carry out the required workload execution. (Luksa 2017, Burns et al., 2022).

Worker nodes are the physical or virtual machines in a Kubernetes cluster that execute the application workloads in the form of Pods and their containers. Whereas the control plane is responsible for managing the cluster and making orchestration decisions, worker nodes provide the processing power, memory, storage, and network connectivity needed for workload execution. Each worker node is designed to receive Pods that have been scheduled by the control plane and to supply the local runtime environment required for those Pods to operate correctly. To achieve this, a worker node runs several essential node-level components, including the kubelet, the container runtime, and kube-proxy, which together ensure that containers are started, maintained, and made accessible over the cluster network. In practical terms, worker nodes are the part of the cluster where the actual business applications run, meaning that although they are directed by the control plane, they are responsible for carrying out the computational work of the system. Consequently, worker nodes form the execution layer of Kubernetes and play a central role in

transforming the desired cluster state defined by the control plane into running application services (Luksa 2017, Burns et al., 2022).

The kubelet is a daemon (node agent) that runs on every worker node and is responsible for everything running on that specific machine. It communicates with the Kubernetes API server to register the node and watch for Pods (groups of containers) that have been scheduled to it. Once a Pod is assigned, the kubelet instructs the container runtime (such as Docker or rkt) to pull the necessary container images and start them. Furthermore, it continuously monitors the running containers, performs health checks (like liveness probes), restarts containers if they crash, and reports their status and resource consumption back to the API server (Luksa 2017, Burns et al., 2022).

The Kubernetes Service Proxy (kube-proxy) is a process that also runs on every node in the cluster. Its primary role is to route network traffic and ensure that clients can successfully connect to the load-balanced services defined in the cluster. It generally does this by configuring the node's network rules (such as Linux *iptables*) to intercept network packets destined for a service's virtual IP address and redirecting them to a randomly selected backend pod that provides that service (Luksa 2017, Burns et al., 2022).

To facilitate this workload execution, the kubelet relies on two fundamental standardized interfaces: the Container Runtime Interface (CRI) and the Container Network Interface (CNI) (Kubernetes 2026a). The CRI is a plugin interface that allows the kubelet to utilize a wide variety of container runtimes—such as Containerd or CRI-O—without requiring the orchestrator to be recompiled. Containerd, for instance, acts as an industry-standard core runtime that securely manages the complete container lifecycle of its host system, from image transfer to execution (Kubernetes 2026b). Simultaneously, the CNI dictates how network interfaces are configured for Linux containers. Plugins implementing the CNI, such as Flannel, are responsible for dynamically inserting network interfaces into the container network namespace and establishing the overlay network that permits seamless pod-to-pod communication across disparate physical worker nodes (Kubernetes, 2024).

2.2.2 Helm

While Kubernetes natively manages deployments through declarative YAML manifests, manually authoring and maintaining these files for dozens of interconnected microservices is highly inefficient and prone to configuration drift. To abstract this complexity, package managers like Helm have become standard in the cloud-native ecosystem (Helm, n.d.). Helm allows engineers to define applications as "Charts," which are collections of files that describe a related set of Kubernetes resources. Rather than hardcoding static deployment parameters, Helm utilizes a templating engine. The static templates are combined dynamically with a `values.yaml` file at runtime, enabling developers to inject specific environment variables, resource limits, and container image tags without altering the underlying infrastructure code (Helm, n.d.). This templated approach is essential for continuous delivery pipelines, as it allows automation servers to programmatically update single configuration values (such as an image version) to trigger complex, multi-service cluster deployments.

Beyond simple templating, the architectural evolution of Helm itself highlights the industry's growing emphasis on cluster security and role-based access control (RBAC). In its earlier iterations (Helm 2), the package manager relied on a two-part architecture: a client-side CLI tool and a highly privileged, in-cluster server component known as "Tiller." While Tiller facilitated state management, it required extensive cluster permissions, presenting a significant security vulnerability in multi-tenant environments (Helm 2019a, Kyverno 2024). Recognizing this fundamental flaw, the release of Helm 3 completely eliminated the Tiller component, transitioning to a purely client-side architecture. Modern Helm relies entirely on the native Kubernetes API and the executing user's defined `kubeconfig` credentials to push changes. This architectural shift significantly hardens the security posture of CI/CD pipelines, ensuring that deployment tools adhere strictly to the principle of least privilege without requiring a persistent, high-risk agent running inside the cluster (Helm 2019a).

A critical differentiator between utilizing raw Kubernetes YAML files and employing Helm is the concept of "Release Management." When a Helm chart is deployed, it creates an immutable instance known as "Release." Helm inherently tracks the state and revision history of every Release by storing metadata directly within Kubernetes secrets (Helm n.d.). Because Helm views a complex microservice application—which may consist of dozens of ConfigMaps, Deployments,

and Services—as a single, atomic unit, it simplifies lifecycle management. If a continuous delivery pipeline deploys a defective update that bypasses testing gates, operators or automated recovery scripts can execute a helm rollback command. This instantly reverts the entire application ecosystem to a previously known, stable state. In the context of automated deployments, this atomic tracking drastically reduces the Mean Time to Recovery (MTTR) during catastrophic failures (Helm, n.d., HashiCorp, n.d.).

To support continuous deployment at scale, infrastructure-as-code must be version-controlled and distributed with the same rigor as compiled application binaries. Helm achieves this through the use of Chart Repositories. Once a Helm Chart is developed, it can be packaged into a versioned, immutable artifact (a .tgz archive) and hosted on remote servers (Helm, n.d.). Recently, the Kubernetes ecosystem has adopted the Open Container Initiative (OCI) standard for Helm charts. This allows engineering teams to store their infrastructural templates in the exact same remote registries (such as Docker Hub or Harbor) utilized for their application container images (Argo CD n.d., Harbor n.d.). Consolidating both application code and infrastructure configurations into a unified registry pipeline streamlines the CI/CD architecture and ensures strict version parity between the software and the environment it runs within.

Finally, while Helm is traditionally executed via a command-line interface as a "push" tool, its true potential is unlocked when integrated with declarative GitOps controllers like Argo CD. In a GitOps architecture, the continuous delivery agent does not execute standard helm install or helm upgrade commands directly (Argo CD n.d.). Instead, controllers natively utilize Helm's templating engine to locally "render" the manifests based on the exact variable values defined in the Git repository. Argo CD then calculates the differential between these newly rendered manifests and the live cluster state, applying only the necessary changes (Argo CD n.d.). This synthesis of Helm's dynamic templating and GitOps' strict reconciliation loops provides development teams with the ultimate flexibility to deploy modular applications while maintaining absolute, auditable control over the cluster's declarative state.

2.3 DevOps Methodologies and CI/CD Principles

DevOps culture emerged to solve the wicked problem of building secure, resilient, and rapidly evolving distributed systems at scale by breaking down traditional silos (Kim et al., 2016, Forsgren et al., 2018). The primary goal of DevOps is to merge software development with IT operations, creating a collaborative environment that enables a fast flow of work and feedback from development to operations to the customer (Bass et al., 2015, Kim et al., 2016). Building upon these core objectives, DevOps represents a fundamental cultural shift that integrates development, quality assurance, and IT operations to facilitate a lean, fluid connection across the entire software lifecycle (Ebert et al., 2016). To achieve this fast flow and rapid feedback, DevOps relies heavily on high degrees of automation, standardization, and the concept of continuous delivery, ensuring that new software versions can be delivered frequently while guaranteeing correctness and reliability (Liu et al., 2021). This automation is typically realized through a deployment pipeline—which encompasses stages like continuous integration, automated acceptance testing, capacity testing, and release—and by treating infrastructure as code rather than relying on manual provisioning (Greising et al., 2018). Ultimately, by reshaping organizational culture and utilizing these automated processes, organizations can successfully reduce the lead time between software development and operational deployment without negatively affecting product quality (Erich et al., 2017, Liu et al., 2021).

Within this culture, delivery pipelines are governed by three distinct but related paradigms. These paradigms are continuous integration, delivery, and deployment. These essentially comprise the software development industry practices that enable organizations to frequently and reliably release new features and products (Shahin et al., 2017).

Continuous Integration (CI): A practice where members of a team integrate their work frequently—usually multiple times a day. Each integration is verified by an automated build and test sequence to detect errors as quickly as possible (Duvall et al., 2007).

Continuous Delivery (CD): Builds upon CI by ensuring that the software is always in a deployable state throughout its lifecycle. In this model, the system can be deployed to production or end users at any time on demand, but the final release decision remains a manual trigger (Humble and Farley, 2010).

Continuous Deployment: The most advanced iteration, where the process of deploying the software into the production environment is completely automated (Humble and Farley, 2010). Any change that successfully passes the automated testing phases is automatically pushed into production without human intervention.

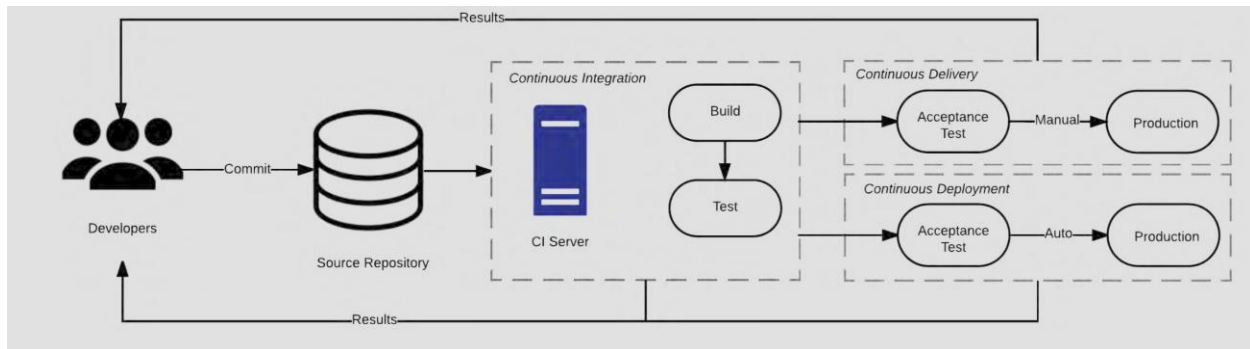


Figure 4: This diagram illustrates a standard software development pipeline where developer commits, trigger automated build and test processes within a continuous integration server. It further contrasts continuous delivery, which requires a manual step to release code to production, with continuous deployment, where the entire release process is fully automated (Shahin et al., 2017).

Research emphasizes that implementing CI/CD pipelines significantly reduces "integration hell"—the painful and time-consuming process of cobbling together software components at the end of a milestone (Duvall et al., 2007). By replacing manual, error-prone tasks with automated assembly lines, CI/CD pipelines accelerate time-to-market, increase deployment frequency, and enable developers to catch and fix defects immediately (Forsgren et al., 2018).

A core tenet of modern DevOps methodologies is the "Shift-Left" testing philosophy, which advocates for moving integration, performance, and security testing as early in the software development lifecycle as possible (Microsoft, 2022, IBM, 2023). Historically, End-to-End (E2E) testing—which validates the entire application workflow from the user interface down to the backend datastores—was reserved for post-deployment staging environments. However, discovering regressions late in the pipeline significantly increases the cost and latency of remediation. To shift these quality gates left, CI/CD pipelines increasingly rely on automated browser testing frameworks, such as Playwright, to validate code immediately upon integration (Playwright, n.d., Microsoft, 2022).

A critical requirement for reliable E2E testing is "Test Idempotency"—the principle that a test must yield the exact same result regardless of how many times it is executed. Achieving idempotency

requires a pristine, unpolluted environment for every test run to prevent false negatives caused by residual data or state conflicts (Hummer et al., 2017, Carlspring, 2024). Maintaining permanent, static staging clusters for this purpose is economically inefficient and highly susceptible to state pollution when multiple developers execute concurrent tests. To resolve this, modern pipelines utilize "Ephemeral Environments." By leveraging localized orchestration tools like Docker Compose, the CI server can dynamically synthesize a temporary, fully functional replica of the entire microservice topology strictly for the duration of the test (Docker 2024, Ephemeral Environments, n.d.). Once the automated assertions are complete, the environment is systematically destroyed, ensuring absolute test isolation and zero infrastructural footprint (Ephemeral Environments, n.d., GitHub Marketplace, 2022).

The "Shift-Left" testing paradigm fundamentally rearchitects the traditional software development lifecycle (SDLC) by relocating quality assurance (QA) processes to the earliest possible stages of code integration (IBM 2023, Rani 2023). In legacy waterfall methodologies, validation occurred downstream, typically in post-deployment staging environments. This delayed feedback loop resulted in an exponential increase in the cost and complexity of remediating defects, as bugs were deeply embedded into the application's architecture before discovery. By executing rigorous automated testing during the continuous integration build phase—effectively "shifting" the tests to the "left" of the deployment timeline—development teams achieve immediate feedback. This proactive mitigation ensures that defective logic, user interface regressions, and broken microservice dependencies are intercepted and isolated before they can pollute the production cluster (IBM 2023, SonarSource 2025).

In distributed microservice architectures, relying solely on isolated unit tests or service-level integration tests is fundamentally insufficient. Because microservices rely on asynchronous network communications and distributed datastores, isolated tests cannot guarantee that the collective system functions cohesively (Bunnyshell 2026, TestRigor 2025). The true operational viability of a distributed architecture can only be verified by executing automated End-to-End (E2E) tests. E2E automation simulates real user workflows—such as navigating a graphical interface, manipulating shopping carts, and executing transactions—thereby tracing the request as it traverses the entire network topology from the ingress gateway down to the backend databases.

Successfully passing an E2E suite serves as the ultimate functional quality gate for a CI/CD pipeline (IBM 2025, Bunnyshell 2026).

To facilitate this rigorous E2E validation within the transient, headless environments of a CI server, the industry has transitioned away from legacy WebDriver-based automation tools (such as Selenium) toward modern, asynchronous frameworks like Microsoft Playwright (Playwright, n.d.). Legacy frameworks historically relied on synchronous HTTP requests to translate test commands into browser actions, introducing severe latency and causing "flaky" tests that frequently timed out in resource-constrained CI environments. In contrast, Playwright utilizes a bidirectional WebSocket connection to communicate directly with the browser engine's internal architecture. This event-driven architecture enables the execution of highly concurrent, cross-browser smoke tests (spanning Chromium, Firefox, and WebKit) entirely in headless mode, making it exceptionally well-suited for automated continuous integration pipelines (Playwright n.d., Mometric 2026).

Furthermore, Playwright incorporates deterministic "auto-waiting" mechanisms and advanced network interception capabilities, which directly address the instability of automated testing in ephemeral environments (Playwright, n.d.). In a dynamic Kubernetes or Docker-Compose test cluster, microservices may exhibit variable initialization and response latencies. Rather than relying on rigid, hardcoded sleep timers, Playwright dynamically interrogates the Document Object Model (DOM) and autonomously waits for elements to become visible, stable, and actionable before executing interactions. Coupled with its ability to intercept, monitor, and mock network traffic between the frontend and backend services, this framework guarantees a high degree of test idempotency and drastically reduces the incidence of false-negative pipeline failures (Playwright n.d., Mometric 2026).

2.4 CI/CD Approaches Specific to Kubernetes

Traditional CI/CD approaches rely heavily on "Push-based" deployment models. In these configurations, continuous integration servers (such as Jenkins, Bamboo, or GoCD) build the executable artifacts and then actively push those artifacts and execute deployment scripts directly against the target infrastructure (Bass et al., 2015, Duvall et al., 2007). These centralized CI/CD tools automate the pipeline by pushing packaged code, automating middleware configurations, and copying files onto production servers. In contrast to traditional push models, the modern standard

for Kubernetes is the "Pull-based" GitOps paradigm, utilizing tools like ArgoCD or Flux. In GitOps, the Git repository acts as the single source of truth for the desired declarative state of the infrastructure (Burns et al., 2016). Rather than a CI server pushing changes into the cluster, a software agent running inside the Kubernetes cluster continuously monitors the Git repository and automatically “pulls” the changes, syncing the cluster’s actual state with the desired state in Git (Burns et al., 2022). The theoretical advantage of GitOps tools over traditional CI/CD servers is enhanced security and reliability; the CI server does not need direct administrative access to the Kubernetes cluster, and any manual configuration drift in the cluster is automatically overwritten by the GitOps agent to match the version-controlled repository (Paavola, 2021).

Argo CD is a Kubernetes-native continuous delivery tool that implements the GitOps model, in which the desired state of applications is defined in Git and automatically reconciled with the live state of one or more Kubernetes clusters. It continuously monitors repository changes, compares deployed resources with the declared configuration, and helps detect and correct configuration drift, which improves consistency, traceability, and rollback safety in application delivery. Argo CD supports multiple configuration formats, including plain YAML, Helm charts, Kustomize, and Jsonnet, making it adaptable to different deployment workflows. It also provides a web UI, CLI, and API for managing applications, observing sync status, and controlling deployments across environments, which is especially useful in multi-cluster cloud-native systems. In practice, Argo CD is widely used to separate continuous integration from continuous delivery and to make deployments more automated, auditable, and resilient (Block and Hernandez, 2025, Argo CD Documentation).

Jenkins is an open-source automation server widely used to implement continuous integration (CI) and continuous delivery (CD) in software development pipelines, automating the build, test, and deployment of code changes across diverse environments. It supports thousands of plugins that enable seamless integration with version control systems like Git, build tools such as Maven and Gradle, cloud platforms including AWS and Azure, and container orchestration tools, making it highly extensible for modern DevOps workflows. Jenkins excels in handling complex pipeline definitions through its Pipeline as Code feature, which uses Jenkinsfile scripts to define repeatable, auditable processes stored in source control, thus improving team collaboration and reducing deployment errors. Jenkins can also be effectively connected with Argo CD to create a powerful

hybrid CI/CD solution: Jenkins handles the CI phase by building artifacts and running tests, then pushes container images or manifests to a Git repository, which Argo CD monitors and automatically deploys to Kubernetes clusters using GitOps principles. This integration leverages Jenkins's strengths in build orchestration with Argo CD's declarative deployment model, enabling faster feedback loops, better traceability, and safer rollouts in cloud-native environments (Laster, 2018, Soni 2021).

While the transition to automated CI/CD and GitOps yields significant benefits in deployment velocity, the automation infrastructure itself introduces new operational vulnerabilities. A primary challenge in maintaining complex delivery pipelines is "Pipeline Brittleness" (Ranger, 2025; SquareOps, 2025). In ecosystems where organizations maintain active forks of open-source projects or synchronize large upstream codebases, CI automation scripts often rely on static directory structures and hardcoded build contexts. When upstream developers refactor application architecture or migrate configuration files, these rigid CI scripts inevitably fail. Consequently, robust pipelines must incorporate defensive programming principles—such as dynamic file-existence validations—to autonomously adapt to structural codebase mutations (Dev.to, 2026; GitLab, 2024).

Furthermore, localized continuous integration environments face severe infrastructural challenges related to resource exhaustion. The repetitive, programmatic compilation of multi-stage Docker images and the orchestration of full-stack ephemeral testing fleets generate massive amounts of volatile data. If left unmanaged, orphaned containers, redundant network bridges, and dangling image layers will rapidly deplete the host node's storage capacity, culminating in fatal pipeline crashes (Docker, 2026; Middleware, 2026). To ensure long-term stability, continuous integration architectures must implement aggressive, automated hygiene routines—often codified as declarative post-execution cleanup blocks—to systematically reclaim computational resources and maintain the operational integrity of the runner node.

3. Methodology

3.1 Infrastructure & Deployment

To accurately simulate a production-grade distributed system environment, a local bare-metal-style Kubernetes cluster was provisioned on a single host machine using Oracle VirtualBox as the virtualization platform. The environment was deployed on a Microsoft Windows 11 host and consisted of three Ubuntu Server virtual machines forming the Kubernetes cluster: one Control Plane (Master) node and two Worker nodes. This topology explicitly satisfies the multi-node requirement necessary for distributed deployment and testing. According to the infrastructure design, the master node was allocated 2 vCPUs and 6 GB of RAM, while each worker node was allocated 4 vCPUs and 6 GB of RAM, providing sufficient computational resources for cluster management and workload execution. Figure 5 illustrates the infrastructure architecture of the experimental environment.

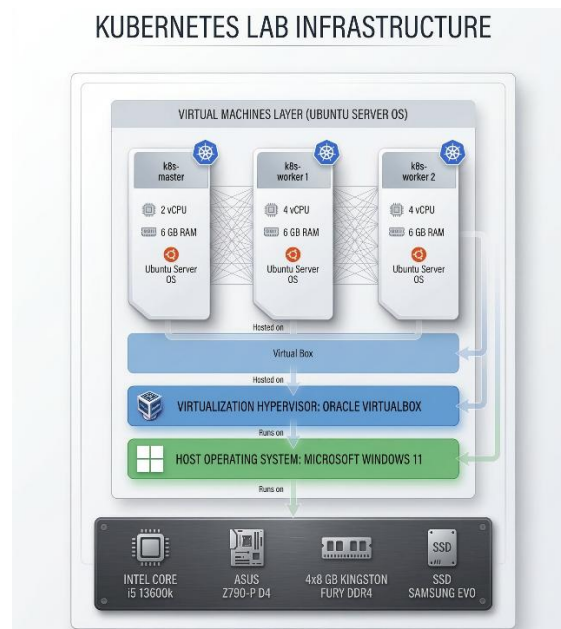


Figure 5: Kubernetes infrastructure.

The operating systems of the Ubuntu virtual machines were specifically tuned to meet strict Kubernetes deployment prerequisites. Swap memory was permanently disabled, which is a fundamental requirement for the kubelet daemon to accurately enforce container resource allocation and maintain predictable scheduling heuristics. Additionally, specific Linux kernel modules were activated to support the container runtime and cluster networking: the overlay

module was loaded to provision the overlay filesystem utilized by Containerd, while the `br_netfilter` module was enabled to ensure that bridged IPv4 traffic is correctly intercepted by iptables, thereby facilitating seamless pod-to-pod communication and accurate Service routing. Furthermore, Containerd was selected and configured as the Container Runtime Interface (CRI), integrating tightly with the underlying system for process management. The cluster was bootstrapped using the `kubeadm` utility, and the Control Plane was initialized with a specific Pod CIDR (10.244.0.0/16) to prevent IP collisions with the host's physical network. Once the worker nodes successfully joined the cluster, Flannel was deployed as the Container Network Interface (CNI) to establish the internal overlay network, allowing seamless and secure communication across the virtualized hardware.

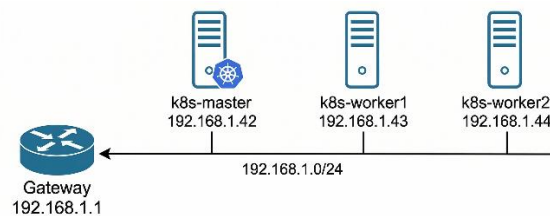


Figure 6: Cluster network configuration.

3.2 Repository Architecture and Connectivity

The core focus of this thesis is the transition from standard manual deployment methodologies to an automated, declarative GitOps approach. To achieve this, a strict separation of concerns was established using a multi-repository architecture. The ecosystem relies on three distinct GitHub repositories: the [Application Source Code repository](#) containing the microservice logic and Dockerfiles, the [GitOps Configuration repository](#) containing the declarative Helm charts and `values.yaml` files, and the [E2E Testing repository](#) containing the Playwright testing frameworks. Complementing these source control environments is a centralized Docker Hub container registry, which functions as the primary [artifact repository](#) for securely storing and distributing the compiled container images (All repositories are referenced in the Appendix section).

This decoupling strictly regulates the flow of continuous integration and continuous delivery. The Jenkins automation server is granted read access to the Application and Testing repositories to pull code and execute builds. It securely authenticates with [Docker Hub](#) using injected credential variables (`DOCKER_CREDS`) to push compiled images. For the final delivery phase, Jenkins utilizes a classic GitHub Personal Access Token (PAT) to gain write access to the isolated GitOps

Configuration repository. By keeping the repositories strictly separated, the architecture ensures that CI processes do not accidentally overwrite production configurations and that the continuous delivery controller (Argo CD) only reacts to explicit, validated configuration changes rather than raw application code commits.

3.3 Baseline Deployment

The initial implementation objective was to deploy a functioning baseline of the Google Online Boutique, a microservices reference architecture comprising eleven distinct components. To facilitate the subsequent implementation of the GitOps pipeline, the official source code repository was first forked into the isolated [Application Source Code repository](#). The unmodified baseline application was then deployed to the cluster using Helm, a Kubernetes package manager, which processed the provided [values.yaml](#) and template files to dynamically generate and apply the necessary Kubernetes manifests to the API server.

The baseline application was successfully exposed via a NodePort service, verifying end-to-end functionality and establishing the core ingress pathway. As illustrated in Figure 7, external client requests—alongside simulated stress traffic generated by a dedicated loadgenerator client—are routed through this NodePort Service directly to the primary frontend microservice. The frontend acts as the central hub of the architecture, utilizing HTTP and gRPC protocols to asynchronously fan out requests to a complex web of backend components. The resulting deployment topology demonstrates a balanced distribution of workloads across the cluster's physical infrastructure. Worker Node 1 hosts the primary ingress components, ancillary services, and the stateful redis-cart datastore, while Worker Node 2 is provisioned to handle the bulk of the transactional processing through components like the checkoutservice, paymentservice, and recommendation service. Furthermore, the diagram visually maps the intricate inter-service dependencies required to process user actions—such as the checkoutservice acting as a secondary orchestrator to coordinate shipping, currency, and payment validations. This detailed visualization establishes a crucial baseline understanding of the application's network topology and internal communication flow, which is essential for evaluating system performance during subsequent CI/CD and stress-testing phases.

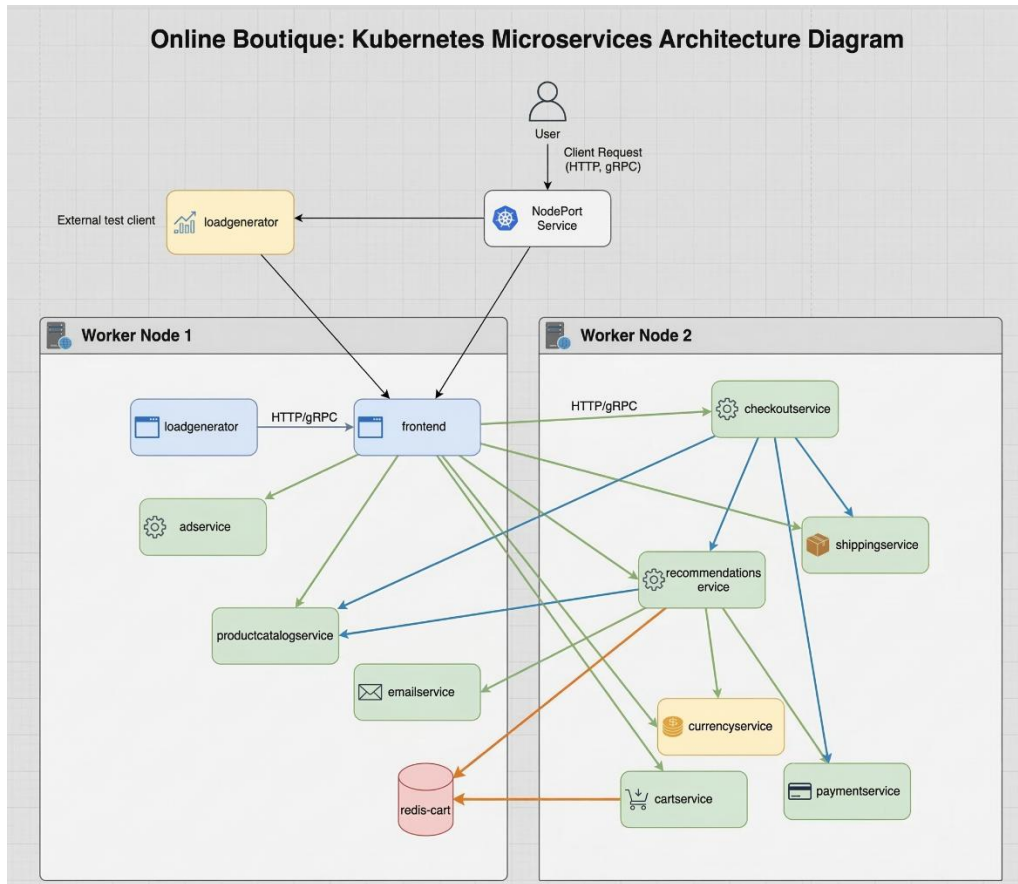


Figure 7: Baseline network topology and service distribution of the Online Boutique microservices architecture. The diagram illustrates the ingress traffic flow via a NodePort Service and the inter-node gRPC/HTTP communication paths across the Kubernetes worker nodes.

3.4 Automated CI/CD Pipeline and Shift-Left Testing Analysis

3.4.1 Intelligent Change Detection and Local Build Phase

The core orchestration of the continuous integration and continuous delivery workflow is codified within a declarative [Jenkinsfile](#) (the complete Groovy source code is available in the GitOps repository referenced in the Appendix). As visually mapped in the architectural diagram (Figure 8), this configuration structures the pipeline into six interconnected stages fortified by a mandatory post-execution cleanup routine, dictating the precise interactions between the CI server and the isolated repositories. The execution environment is initially configured by securely injecting authentication credentials for Docker Hub and GitHub, alongside defining the target repository URLs. During the "Checkout & Smart Detect" stage, the pipeline clones the application repository and implements a critical loop-prevention mechanism by analyzing the latest commit author; if the commit is identified as an automated GitOps update from the "Jenkins CI Server," the build is

immediately aborted. The script then executes a differential Git analysis (`git diff --name-only HEAD~1 HEAD`) to parse modifications strictly within the `src/` directory, dynamically aggregating a target list of changed microservices. This list directly dictates the behavior of the "Build Images" stage, where a script loop iterates through the identified services to execute localized docker build commands, assigning a unique version tag derived from the incremental Jenkins build number (`v1.0.${env.BUILD_NUMBER}`). Crucially, the pipeline retains these images locally and transitions into the "Spin Up Ephemeral Environment" stage, utilizing `docker-compose up -d --build` coupled with a brief stabilization delay to boot a temporary, functional replica of the application architecture.

The integrity of this local build is subsequently validated in the fourth stage, where the pipeline clones the dedicated testing repository and instantiates an official Playwright Docker container. By attaching this container to the host network (`--network host`) and injecting the localized frontend URL as an environment variable, the pipeline executes the automated End-to-End test against the ephemeral environment. Serving as a definitive quality gate, the successful completion of these assertions permits the pipeline to advance to the fifth stage, where it securely authenticates with Docker Hub via standard input and pushes the newly validated images to the central registry. In the final "Update GitOps Repo" stage, the pipeline clones the infrastructure configuration repository without triggering changelog polling. It utilizes stream editor (`sed`) commands to programmatically update the target Helm values.yaml file with the newly generated image tags, assigns a generic automation signature to the Git configuration, and pushes the declarative commit back to the main branch to trigger Argo CD synchronization. Irrespective of the pipeline's overall pass or fail status, a post execution block functions as an automated system janitor. This always directive systematically tears down the ephemeral containers (`docker-compose down -v`) and forcefully prunes dangling Docker images to prevent progressive storage exhaustion on the integration server, ensuring a pristine environment for subsequent automated builds.

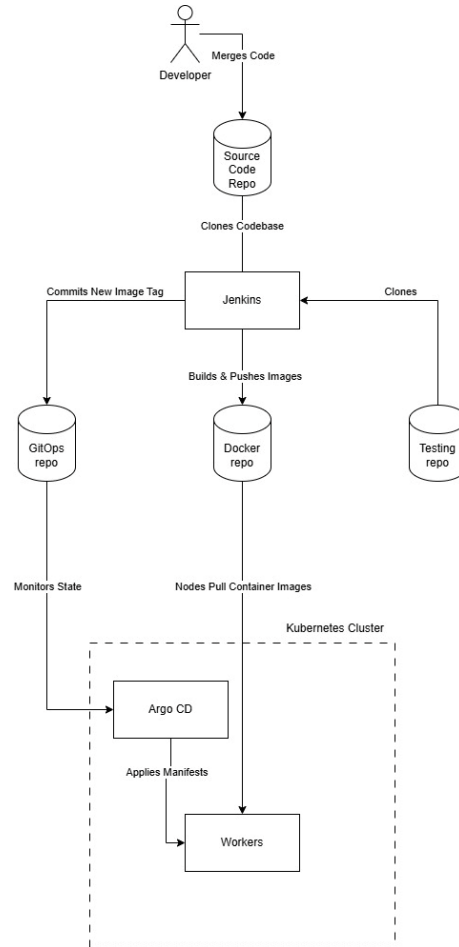


Figure 8: Architectural flow of the GitOps-driven CI/CD pipeline. The diagram illustrates the decoupling of continuous integration (Jenkins) from continuous delivery (Argo CD), highlighting the multi-repository strategy and the "pull-based" deployment mechanism within the Kubernetes cluster.

3.4.2 Ephemeral Environment Instantiation and E2E Testing

Applying the Shift-Left philosophy, the pipeline mandates that code be rigorously verified before it is considered for deployment. Utilizing a local `docker-compose.yaml` configuration, Jenkins provisions a fully isolated, ephemeral testing environment directly on the CI server by executing `docker-compose up -d --build`. This environment perfectly mirrors the production architecture, orchestrating all inter-dependent microservices locally and routing the frontend interface to host port 8081 to avoid network collisions. Once the local fleet is booted, Jenkins pulls the E2E Testing repository and initiates the automated End-to-End (E2E) testing suite, engineered using the Playwright framework. The pipeline executes a specialized, [officially maintained Playwright Docker container](#) tethered directly to the host's network. The testing script initializes by

dynamically resolving its target endpoint, tethered directly to the host's network. The [testing script](#) initializes by dynamically resolving its target endpoint via the `FRONTEND_URL` environment variable. This dynamic binding is critical, as it allows the testing framework to seamlessly target the ephemeral `localhost:8081` environment rather than hardcoding static IP addresses that could lead to environmental conflicts (the complete testing source code is available in the [testing repository](#) referenced in the Appendix).

To guarantee absolute test idempotency, the architecture implements a rigorous before each setup hook. Prior to executing any business logic, the automation engine actively navigates to the localized application instance and interrogates the Document Object Model (DOM) to assess the state of the shopping cart. If residual data or cached items from a previously aborted run are detected, the script programmatically clicks the "Empty Cart" button. This self-cleaning mechanism ensures a sterile, predictable baseline for every single execution, eliminating false negatives caused by polluted test environments.

The core testing sequence simulates a user's purchasing journey to serve as the definitive functional Quality Gate within the CI/CD pipeline. The automation engine navigates the product catalog, interacts with dropdown locators to manipulate item quantities, and performs dynamic data extraction. Rather than relying on static, hardcoded price checks, the script reads the rendered text content of the product price, sanitizes the string using a custom `removeNonDigits` utility function, and mathematically calculates the expected cart total. The framework then executes a strict assertion, verifying that the dynamically calculated string becomes distinctly visible within the updated cart DOM. Subsequent test steps validate the checkout continuum by programmatically injecting synthetic guest email credentials and submitting the transaction payload. The pipeline's ultimate pass or fail status is explicitly bound to deterministic visual assertions, primarily the expectation that the success heading is successfully rendered by the backend (e.g., `expect(page.getByRole("heading", {name: "Your order is complete!" })).toBeVisible()`).

From a technical maintainability perspective, the current script utilizes functional index-based locating strategies (such as `.nth(2)`) and hierarchical CSS child selectors to target specific DOM elements. While these strategies successfully validate the current architecture, they highlight a common trade-off in UI automation; migrating toward explicit, decoupled test identifiers (such as

data-testid attributes) in future iterations would further harden the framework against minor frontend structural changes.

3.4.3 Quality Gate and Defective Code Mitigation

This localized testing phase acts as a strict Quality Gate, directly addressing the objective of system resilience. In the event that defective code is introduced into the source repository—such as a broken DOM element or a failed backend database transaction—the Playwright assertions fail, resulting in a non-zero exit code. The Jenkins pipeline immediately detects this anomaly and halts progression, routing execution directly to the failure block.

Regardless of the test outcome, a mandatory post execution block—referred to as the "Janitor" routine—is triggered. This sequence executes `docker-compose down -v` and `docker image prune -f`, safely tearing down the ephemeral environment and purging temporary images to prevent storage exhaustion on the CI host. If the pipeline halted due to test failure, the defective images are permanently discarded before reaching the deployment phase, guaranteeing that erroneous configurations never pollute the production Kubernetes cluster and ensuring near-instantaneous feedback latency for developers.

3.4.4 Continuous Delivery via Argo CD (GitOps)

Upon the successful passing of all automated assertions, the pipeline proceeds to the continuous delivery phase. Jenkins securely authenticates with Docker Hub and executes the `docker push` commands for the locally cached, now-validated images. Subsequently, the CI server automatically clones the [GitOps Configuration repository](#) without triggering changelog polling. Utilizing a stream editor utility (`sed`), the pipeline dynamically updates the image: strings within the target [values.yaml](#) file to reflect the new, validated build tags. The Jenkins server then signs the commit as an automated user and pushes the configuration changes back to the main branch of the GitOps repository.

At this juncture, the deployment architecture transitions entirely from a push model to a declarative GitOps pull model. Argo CD, operating as an autonomous controller within the production Kubernetes cluster, continuously monitors the GitOps repository—which acts as the absolute single source of truth. Upon detecting the state drift injected by Jenkins, Argo CD automatically synchronizes the cluster state. It translates the updated Helm templates into raw Kubernetes

manifests and orchestrates a zero-touch rolling update across the worker nodes. This decoupled approach ensures high availability, abstracts administrative credentials away from the CI server, and maintains a strictly declarative «version» history, facilitating immediate and flawless rollbacks in the event of post-deployment degradation.

4. Experimental Study

The core objective of this experimental study was to evaluate the resilience, automation capabilities, and protective mechanisms of the proposed declarative GitOps architecture. To empirically validate the system, the architecture was subjected to four distinct operational scenarios. These scenarios simulated real-world software development lifecycles, ranging from successful feature enhancements to catastrophic architectural failures and unauthorized infrastructural tampering. The results of these experimental runs, executed via the Jenkins CI pipeline and monitored by the Argo CD continuous delivery controller, are detailed below (The Jenkins output for each scenario can be found [here](#)).

4.1 Scenario 1: Safe Feature Enhancement (Continuous Delivery Validation)

The first scenario validated the pipeline's ability to facilitate rapid, zero-touch continuous delivery for safe code changes. A benign feature enhancement was introduced to the Application Source Code repository by modifying the frontend microservice's HTML templates to display a new promotional banner (`<p>Thesis Scenario 1: Automated Deployment Successful</p>`) and an updated header (`<h3>Hot Products in this site TESTING</h3>`). Figure 9 shows the current state of the website.

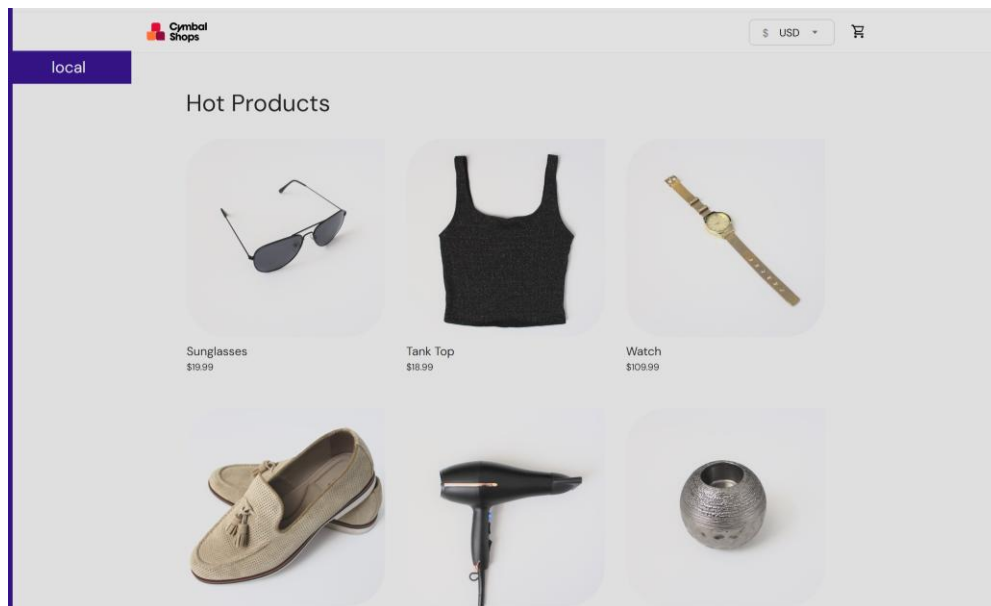


Figure 9: Current state of the website before the changes.

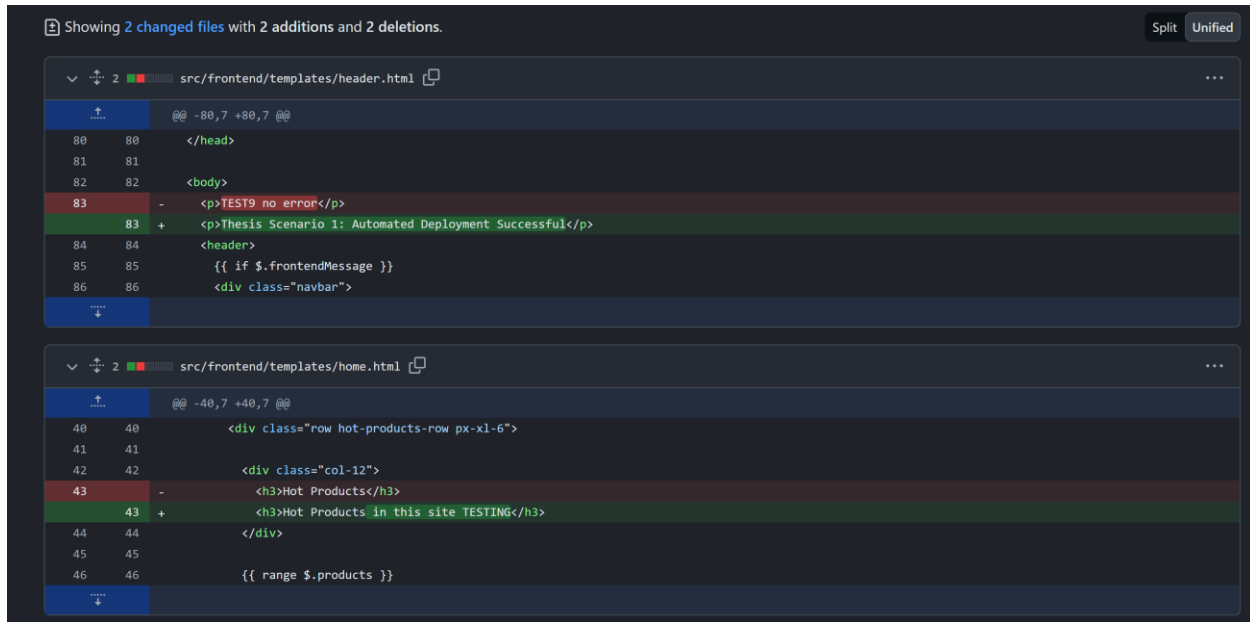


Figure 10: Changes shown in the GitHub before merge.

Upon committing the change (as shown in Figure 10), the Jenkins CI server automatically triggered build #192 (as shown in Figure 11).

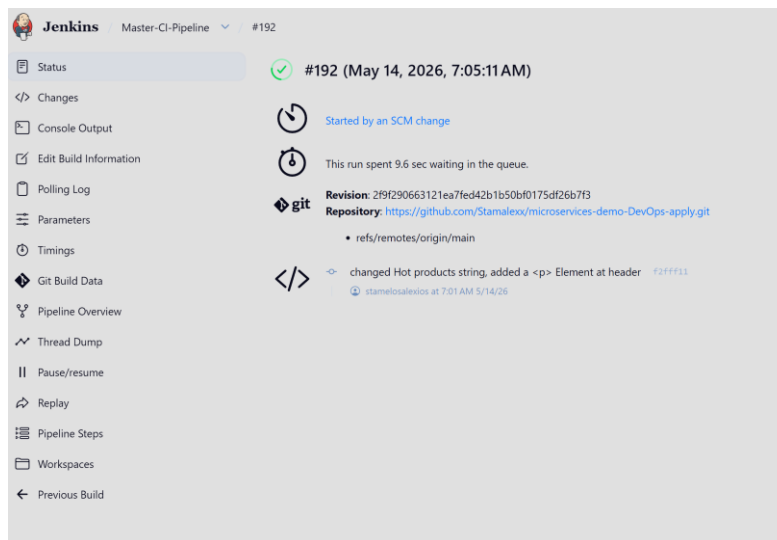


Figure 11: Jenkins detects the changes.

The Jenkins console output can be found in [Thesis_Scenario_1_Automated_Deployment_Successful.txt](#) file. The console output provides a granular, time-stamped record of the Continuous Integration (CI) pipeline's execution during the safe feature enhancement scenario. An analysis of

this execution log empirically validates the multi-stage architecture and the sequential logic of the automated deployment pipeline.

The pipeline initiates by pulling the application source code from the primary repository. Crucially, the log demonstrates the execution of a "Smart Detect" mechanism, utilizing the `git diff --name-only HEAD~1 HEAD` command to intelligently identify the specific microservices modified in the commit. The output explicitly confirms that the system correctly isolated the frontend service (SMART DETECT: Preparing to build -> frontend), ensuring that compute resources were not wasted rebuilding unmodified components.

During the second stage, a localized Docker image is compiled using a secure, multi-stage build process. The execution log records the Golang builder compiling the application binary before transferring the compiled artifact to a minimal, highly secure `gcr.io/distroless/static` base image. This process successfully generates the localized artifact tagged as `stamalexx/frontend:v1.0.192`.

To facilitate integration testing without impacting production, the CI server provisions a fully isolated testing environment directly on the runner node using `docker-compose up -d --build`. The console output meticulously records the sequential creation, initialization, and starting phases of the entire microservice fleet, including infrastructural components like the redis-cart datastore. To prevent race conditions, the pipeline enforces a mandatory 30-second delay (`sleep 30`), ensuring all inter-service network connections are fully established before testing begins.

Once the environment is stabilized, the pipeline clones the dedicated E2E testing repository. It then instantiates an official, pre-configured Microsoft Playwright testing container (`mcr.microsoft.com/playwright:v1.59.1-jammy`) bound to the host network. The log indicates that the test suite targets the dynamically assigned `http://localhost:8081/` endpoint. The smoke tests are executed concurrently across three distinct browser rendering engines (Chromium, Firefox, and WebKit), successfully validating the user journey (e.g., adding items to the cart and placing an order) with a 100% pass rate in 42.2 seconds.

Having successfully passed the rigorous E2E quality gate, the pipeline executes the registry synchronization. The Jenkins server securely authenticates with Docker Hub and pushes the validated `v1.0.192` image to the remote `stamalexx/frontend` repository.

In the final operational stage, the pipeline bridges the gap between Continuous Integration and Continuous Delivery. Jenkins clones the declarative GitOps Configuration repository. It utilizes a sed stream editor command to dynamically modify the helm-chart/values.yaml file, overwriting the previous image version with the newly validated v1.0.192 tag. The Jenkins CI Server then automatically commits and pushes this configuration change back to the repository ([main da416d7] Auto-Deploy: Updated [frontend] to v1.0.192). This automated commit serves as the final action of the CI pipeline, effectively shifting the responsibility to the Argo CD controller to recognize the state drift and execute the deployment.

Following the GitOps handoff, the pipeline executes a declarative post-action block to maintain the operational hygiene of the CI server. The ephemeral environment is systematically dismantled using docker-compose down -v, which stops and removes all local containers and networks. Finally, a docker image prune -f command is executed to purge dangling images, successfully reclaiming 914.9MB of disk space, ensuring the runner remains optimized for future builds.

Within seconds, the Argo CD continuous monitoring loop detected the configuration drift in the Git repository. Acting as the active reconciliation engine, Argo CD synchronized the cluster state (as shown in Figure 12 and 13), successfully rolling out the new frontend pods without downtime. The code was successfully merged in the deployed environment (as shown in Figure 14). The entire automated lifecycle—from code commit to live production deployment—concluded successfully within one minute, empirically proving the efficiency of the "Push-to-Pull" GitOps methodology.

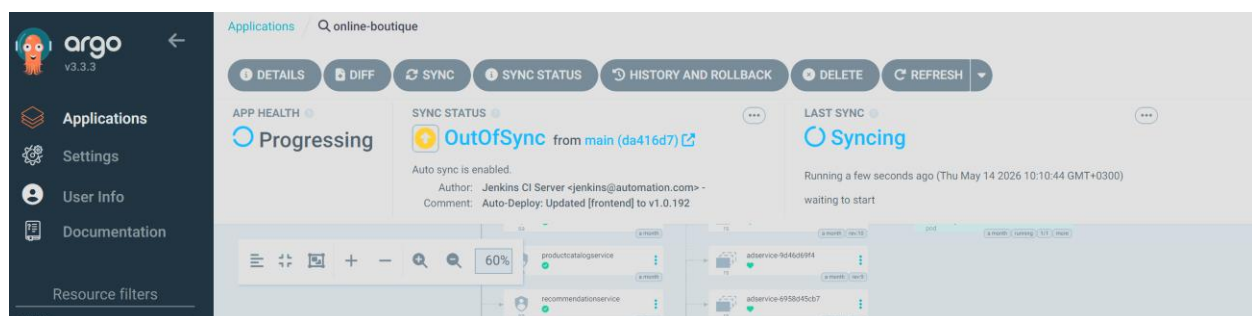


Figure 12: Out of sync state of the app.

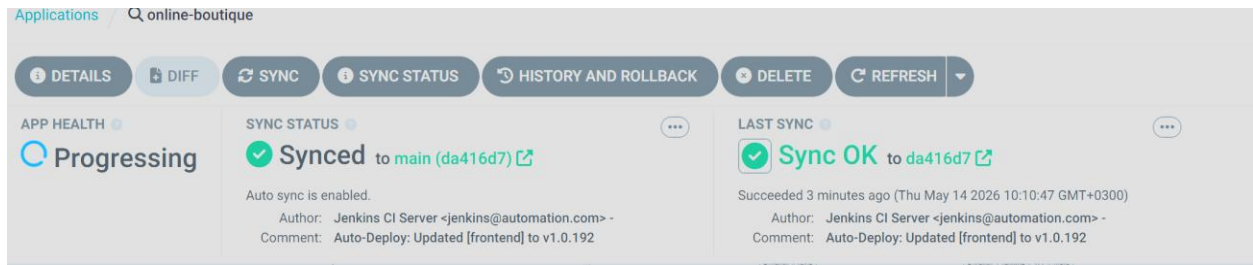


Figure 13: Synced state of the app.

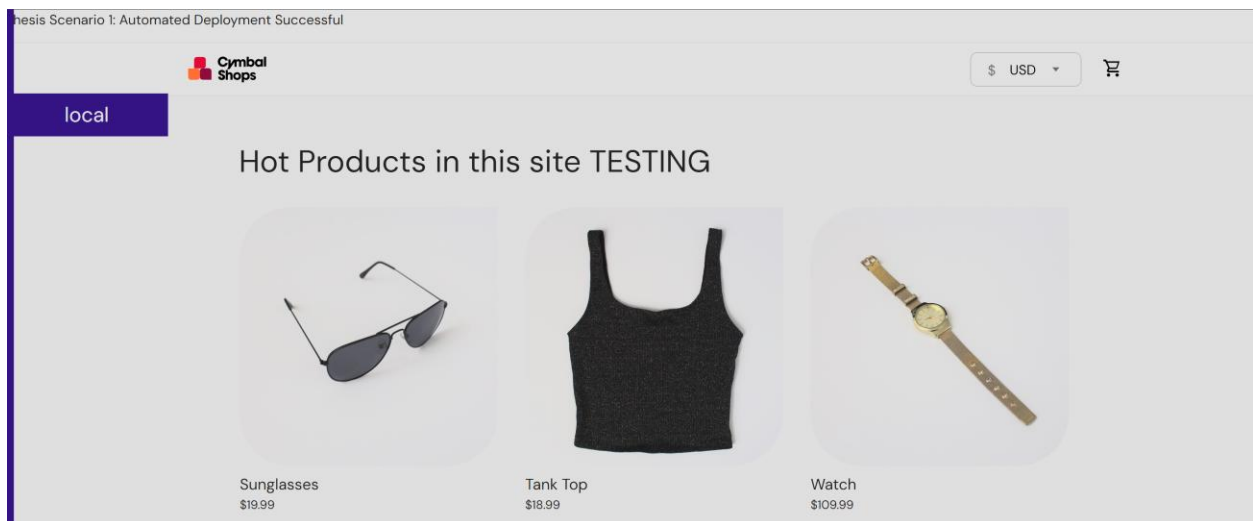


Figure 14: Deployed application after the changes.

4.2 Scenario 2: UI Regression Detection (Shift-Left Quality Gate)

The second scenario tested the architecture's ability to intercept defective user interface logic before it could impact the production environment. A simulated regression was introduced by altering the cart.html template in the frontend service, changing the primary checkout button text from "Place Order" to "Pay Now" (Figure 15 shows the changes in the code). This simulated a developer failing to update the corresponding automation tests alongside a UI overhaul.

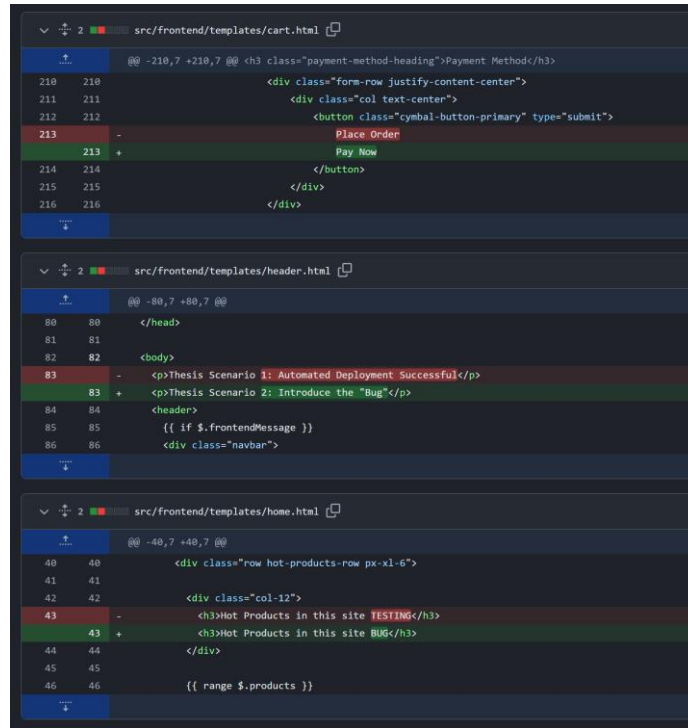


Figure 15: Changes shown in the GitHub before merge.

Jenkins build #193 initiated the pipeline. The system successfully compiled the code and provisioned the ephemeral test environment. The superiority of the automated End-to-End (E2E) testing framework became evident during this validation phase. The pipeline executed the Playwright testing container, which launched concurrent smoke tests across Chromium, Firefox, and WebKit browser engines. During the user journey simulation, all three testing instances encountered a critical failure when attempting to finalize a purchase. The console output meticulously records a fatal timeout error (Test timeout of 30000ms exceeded) because the automation engine was unable to locate the expected DOM element (waiting for `getByRole('button', { name: 'Place Order' })`). This explicit failure confirms that the test suite successfully identified the missing or altered UI component dynamically at runtime.

In direct response to the test suite's non-zero exit code, the pipeline's protective mechanisms were immediately triggered. The execution log confirms that the critical deployment stages—pushing the artifact to Docker Hub and updating the GitOps configuration repository—were actively bypassed due to the preceding failure (the logs can be found in the [Thesis_Scenario_2_The_bad_scenario_Introduce_the_Bug.txt](#)). The pipeline then automatically routed to its declarative post-action block, completely dismantling the ephemeral environment (`docker-compose down -v`)

and reclaiming 914.9MB of disk space by pruning the localized images. The process concluded with a formal pipeline failure state and the explicit log notification: "PIPELINE FAILED: Defective code detected! Deployment to GitOps was blocked.". This provides definitive empirical proof that the CI/CD architecture successfully intercepts regressions and physically isolates the production cluster from defective merges. As shown in Figure 16, Jenkins did not push the new image to the docker repository, therefore ArgoCD did not sync.

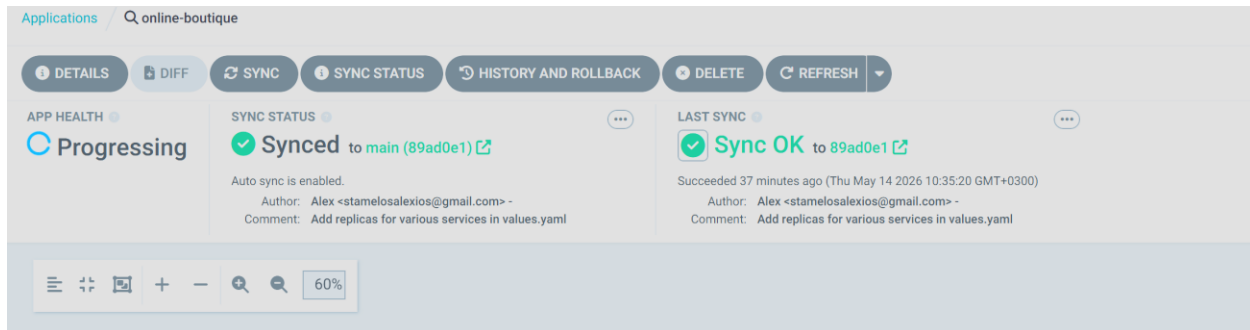


Figure 16: Sync state of the application. Jenkins did not push the new image, therefore ArgoCD did not sync.

4.3 Scenario 3: Inter-Service Dependency Failure (Catastrophic Crash Prevention)

Scenario 3 escalated the testing parameters by introducing a fatal architectural error designed to evaluate the pipeline's ability to catch deep, backend dependencies. A fatal JavaScript exception (`throw new Error("Thesis Catastrophic Crash Scenario!");`) was injected into the initialization sequence of the Node.js-based `currency` service.

During Jenkins build #195. The execution log for the third operational scenario provides critical empirical evidence of the architecture's ability to detect and isolate fatal, inter-service dependency failures during runtime validation. The Jenkins pipeline was triggered by a source code management event, specifically a commit merging a catastrophic scenario into the repository. The pipeline's Smart Detect mechanism successfully identified the modification within the `currency` service and proceeded to construct the local Docker image. Notably, the Docker daemon successfully built and tagged the `stamalexx/currency: v1.0.197` artifact without error. This is a crucial observation, as it demonstrates that standard continuous integration compilation stages do not execute application logic and are therefore blind to runtime panics or injected fatal exceptions in interpreted languages like Node.js.

The necessity of the "Full-Stack Ephemeral Testing" methodology is explicitly validated during the subsequent stages. The CI server successfully orchestrated the isolated testing fleet via docker-compose, initiating all service containers, including the compromised currency service. However, during the execution of the Playwright End-to-End testing suite, the pipeline encountered a cascading architectural failure. The testing framework attempted to execute the user journey across Chromium, Firefox, and WebKit browsers but failed uniformly across all three engines. The console output meticulously records that the tests failed during the before each initialization hook, throwing a fatal Test timeout of 30000ms exceeded error. The call log explicitly details that the automation engine timed out while waiting for `getByRole('button', { name: 'Empty Cart' })`. Because the frontend service relies on the crashed currency service to render its pricing data, the frontend UI completely failed to load, proving that the pipeline successfully evaluates deep integration dependencies rather than isolated container viability.

In response to this catastrophic testing failure, the pipeline's protective quality gates functioned exactly as designed. The execution log confirms that both Stage 5 (Push to Docker Hub) and Stage 6 (Update GitOps Repo) were explicitly skipped due to the prior testing failures. The pipeline immediately transitioned to the declarative post-action teardown phase, systematically stopping and removing the ephemeral network and containers before pruning dangling images to reclaim 484.8MB of runner disk space. The execution concluded with an exit code of 1 and the explicit log notification: "PIPELINE FAILED: Defective code detected! Deployment to GitOps was blocked.". This provides definitive proof that the CI/CD architecture actively prevents fatal backend microservice crashes from traversing the pipeline and destabilizing the production cluster (the Jenkins log can be found in [Thesis_Scenario_3 The Catastrophic Scenario \(Fatal Microservice Crash\).txt](#)).

4.4 Scenario 4: Unauthorized State Drift (GitOps Self-Healing)

The final scenario bypassed the CI pipeline entirely to validate the autonomous self-healing capabilities of the continuous delivery controller. To simulate a malicious attack or a catastrophic operator error (State Drift), a direct command (`kubectl delete deployment frontend`) was executed against the production cluster, forcefully removing the primary ingress application, as documented in the terminal workspace in Figure 17.

Without requiring any external webhook triggers or human intervention, Argo CD autonomously issued a synchronization command to the Kubernetes API. The missing deployment was instantly regenerated, terminating the drift and successfully restoring the system to a completely healthy, balanced, and synchronized baseline state. This outcome successfully validates the fundamental philosophy of declarative infrastructure: the Git repository acts as the absolute single source of truth, granting the microservices architecture highly resilient, zero-touch recovery capabilities against manual tampering and localized infrastructural degradation.



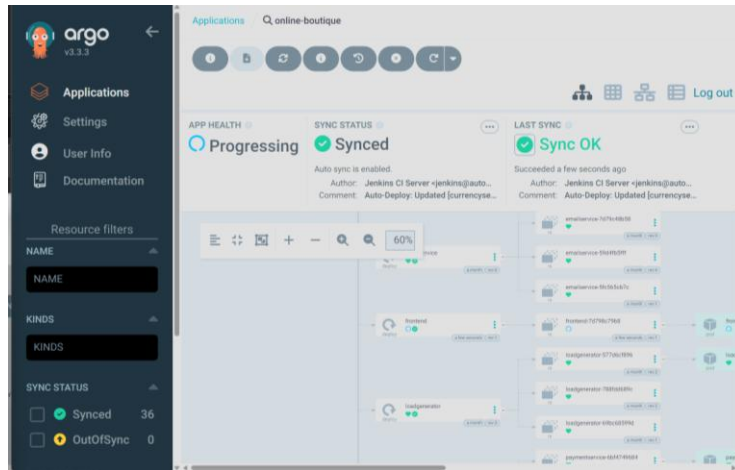


Figure 18: Argo CD continuous delivery dashboard detecting the environmental drift. The application health status transitions to 'Progressing' as the controller automatically synchronizes and regenerates the missing deployment to match the declarative Git repository.

4.5 Scenario 5: Upstream Synchronization and Bulk Integration

The final operational scenario evaluated the architecture's capacity to handle large-scale codebase synchronization. In modern cloud-native development, organizations frequently maintain forks of upstream repositories or rely on distributed teams committing bulk updates. To simulate this real-world integration challenge, 38 asynchronous commits from the original upstream repository (Google Cloud Platform developers) were bulk-merged into the localized repository branch ([Bulk merge commit](#)).

This scenario fundamentally stress-tested the pipeline's dynamic build logic. Upon detecting the massive merge commit, the Jenkins CI server initiated the automated workflow. The "Smart Detect" mechanism successfully analyzed the extensive Git diff spanning the 38 commits, isolating exactly which microservices were modified by the upstream developers (Figure 19). Rather than executing a monolithic rebuild, the pipeline dynamically targeted only the affected localized Docker images to optimize CI compute resources.

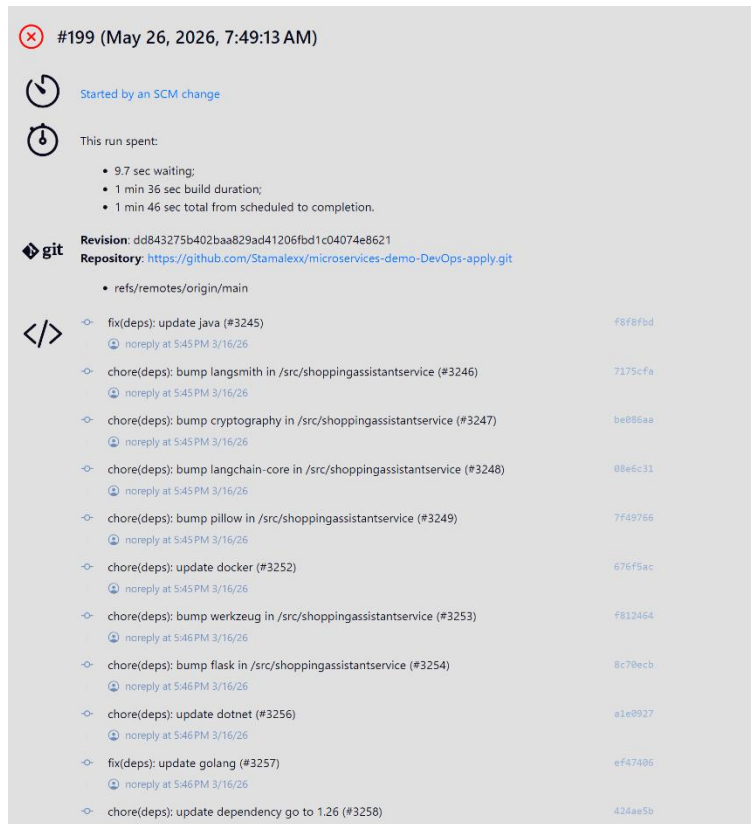


Figure 19: Jenkins detects the changes.

However, during the execution of this bulk-merge, a localized pipeline failure occurred during the artifact compilation stage. The Docker daemon returned a fatal error (unable to evaluate symlinks in Dockerfile path: lstat ... no such file or directory) when attempting to build the cartservice microservice ([Initial failure pipeline console output](#)).

Investigation revealed that this failure was caused by architectural refactoring within the upstream repository. The original developers had migrated the cartservice build context, moving the Dockerfile into a nested subdirectory (src/cartservice/src/). The Jenkins CI pipeline, which initially relied on static directory pathing, failed to locate the file and aborted the build.

This incident provided valuable empirical evidence of "pipeline brittleness"—a common vulnerability in CI/CD architectures where rigid automation scripts break due to upstream repository mutations. To resolve this, the pipeline logic was refactored to incorporate defensive programming principles. File-existence validation algorithms were implemented within the CI script, allowing the pipeline to dynamically scan for the build context across multiple potential directory structures before executing the Docker build command. This adaptation successfully

restored the CI/CD lifecycle, proving that resilient, dynamic scripting is a mandatory requirement for maintaining automated synchronization with active open-source forks ([Refactored pipeline code](#)).

Following the deployment of this updated script, the pipeline was re-triggered to resume the upstream synchronization. While the dynamic pathing successfully resolved the initial build errors, the sheer volume of image compilations required by the massive bulk-merge exposed a secondary, infrastructure-level vulnerability within the localized CI environment. During compilation, the Jenkins runner node encountered a fatal resource exhaustion event (Out of Disk Space). Because the node crashed mid-execution, the pipeline was unable to reach its declarative post-action teardown block, leaving orphaned containers and dangling image layers consuming the remaining storage.

Following minimal manual intervention to restore baseline operational capacity, the failed pipeline state was re-initialized (Build #204). An analysis of the resulting execution log provides critical empirical evidence of the system's operational resilience ([Final successful merge console output](#)). Because the triggering commit encapsulated extensive upstream architectural changes, the pipeline's Smart Detect mechanism aggressively, and correctly, queued all twelve microservices for re-compilation (SMART DETECT: Preparing to build -> adservice, cartservice, checkoutservice...).

Crucially, the console output proves that the defensive pipeline scripting functioned flawlessly at scale. As the pipeline iterated through the microservice fleet, the dynamic file-existence validation successfully mapped the disparate directory structures. The execution log demonstrates the script correctly routing standard services, like adservice and emailservice, to their original build contexts (Found Dockerfile in legacy root directory). Simultaneously, it autonomously detected the upstream refactoring of the C# application, successfully adjusting its routing (Found Dockerfile in nested directory (Refactored Upstream)) before compiling the cartservice. This dynamic adaptation allowed the CI server to compile the entire refactored fleet without encountering a single pathing failure.

Following successful artifact compilation, the CI server orchestrated the isolated docker-compose environment, securely networking the newly synchronized upstream microservices alongside the backend datastores (e.g., redis-cart). The End-to-End automated testing suite was executed against

this highly mutated environment. By passing the Quality Gate, the pipeline empirically confirmed that the massive upstream synchronization introduced no UI regressions or broken inter-service dependencies. Only after this runtime validation did the pipeline securely authenticate and push the artifacts to the container registry.

The root cause of the initial pipeline crash—disk space exhaustion—highlights the intense storage overhead required to build and orchestrate microservice fleets locally. To programmatically mitigate future node degradation, the pipeline relies on its declarative post execution block. The console log confirms that upon successful GitOps handoff, the system autonomously executed `docker-compose down -v` to aggressively dismantle the ephemeral network and containers. This was immediately followed by a forced system prune (`docker image prune -f`), which systematically deleted dozens of dangling, orphaned Docker image layers generated during the multi-stage builds. This automated hygiene mechanism successfully reclaimed critical disk space, proving that aggressive resource management is a mandatory requirement for maintaining long-term infrastructural stability in localized CI/CD environments.

Following the successful compilation, ephemeral validation, and hygiene reclamation phases, the continuous delivery controller autonomously synchronized the production cluster with the updated GitOps repository. The bulk modifications introduced from the upstream branch were successfully integrated and deployed to the live environment. While these upstream commits consisted primarily of underlying codebase refactoring rather than user-facing visual alterations, the successful deployment empirically confirms that the architecture gracefully absorbed the external updates. The production cluster remained perfectly stable, validating that the CI/CD quality gates can effectively manage and deploy large-scale upstream synchronizations without degrading system performance.

5. Conclusions

5.1 Evaluation of the GitOps Paradigm

The primary objective of this applied engineering study was to design, implement, and evaluate the resilience and automation capabilities of a declarative GitOps continuous integration and continuous delivery (CI/CD) pipeline tailored specifically for Kubernetes-based microservices. Unlike traditional monolithic applications, distributed microservices introduce immense deployment complexity due to their intricately coupled network communications and independent release cycles. The results obtained from the five experimental scenarios unequivocally demonstrate that decoupling the continuous integration phase from the continuous delivery phase mitigates these operational risks. By replacing traditional "push-based" server deployments with a "pull-based" GitOps paradigm orchestrated by Argo CD, the architecture successfully abstracted administrative cluster access away from the CI server. This architectural decision not only accelerated the time-to-market for benign feature enhancements, as seen in the first scenario, but also provided a highly auditable, version-controlled mechanism for tracking infrastructure modifications, ultimately fulfilling the core DevOps promise of rapid, reliable software delivery.

5.2 Shift-Left Testing and Inter-Service Validation

Historically, user interface regressions and functional defects were often discovered late in the deployment lifecycle, frequently after code had already breached staging or production environments. The experimental introduction of a simulated UI bug in the second scenario empirically validated that intercepting defective logic during the integration phase physically insulates the production cluster. By mandating that the automated Playwright testing suite executes dynamically against a localized environment, the pipeline effectively established a definitive quality gate. The immediate pipeline termination and bypass of the GitOps repository update upon encountering the timeout error confirms that automated assembly lines can proactively catch human developer errors without requiring manual intervention. This immediate feedback loop drastically reduces the integration latency for development teams, allowing them to rectify defective commits before they compound into larger architectural failures.

Furthermore, the third scenario involving a catastrophic inter-service dependency failure exposed a fundamental limitation in standard continuous integration practices. Traditional CI pipelines

primarily focus on syntax verification and artifact compilation, which are fundamentally insufficient for dynamic, interpreted languages or complex backend communications. The fact that the Docker daemon successfully built and tagged the compromised currency service image without error proves that successful container compilation does not guarantee application viability. It was only through the orchestration of the full-stack ephemeral testing fleet that the cascading failure was detected. Because the primary ingress frontend relies entirely on synchronous internal communications to render its user interface, the injected backend exception triggered a uniform failure across all automated browser engines. This outcome dictates that modern CI/CD pipelines must evaluate runtime behavior and holistic integration dependencies rather than merely verifying isolated container viability, ensuring that deep backend crashes cannot traverse the pipeline.

5.3 Ephemeral Orchestration and Test Idempotency

The methodology employed to facilitate this runtime validation—specifically, the instantiation of a localized, ephemeral container orchestration environment—constitutes a highly optimized paradigm for continuous integration testing. Traditional enterprise architectures often rely on persistent staging clusters, a practice that not only incurs substantial infrastructural overhead but also introduces critical vulnerabilities related to state pollution when accommodating concurrent development branches. By programmatically synthesizing the microservice topology, encompassing even stateful datastores such as Redis, directly upon the continuous integration execution node, the proposed architecture provisions an unadulterated, transient replica of the production environment for each discrete codebase commit.

Furthermore, the integration of deterministic pre-test setup hooks—designed to actively interrogate the Document Object Model (DOM) and programmatically expunge residual transactional data—guarantees strict test idempotency. This active state-cleansing mechanism effectively mitigates the incidence of false-negative validation failures attributable to environmental pollution. Ultimately, this localized orchestration strategy seamlessly bridges the operational delta between isolated developer workstations and the live Kubernetes cluster, delivering high-fidelity integration assurances while systematically negating the financial and operational burdens associated with permanent staging infrastructure.

5.4 Autonomous Reconciliation and Disaster Recovery

Transitioning to the delivery phase, the fourth scenario successfully validated the autonomous self-healing capabilities inherent to the declarative GitOps philosophy. Manual infrastructural tampering, often referred to as state drift, is a pervasive threat in operations, whether caused by malicious actors or catastrophic operator error. The deliberate manual deletion of the primary frontend deployment provided a stark demonstration of Argo CD's active reconciliation loop. Because the Git repository was established as the absolute single source of truth, the continuous delivery controller immediately recognized the deviation between the live cluster state and the declarative configuration. The instantaneous regeneration of the missing infrastructure, executed without any external webhook triggers or human intervention, represents a paradigm shift in disaster recovery. It empirically proves that GitOps grants the microservices architecture highly resilient, zero-touch recovery capabilities, fundamentally reducing the mean time to recovery for localized infrastructural degradation.

5.5 Pipeline Brittleness and Infrastructural Hygiene

Despite the successes of the deployment workflows, the fifth scenario exposed significant integration challenges related to pipeline brittleness during large-scale upstream synchronization. Maintaining active forks of open-source repositories or distributed codebases requires continuous integration logic capable of absorbing massive, unpredictable architectural mutations. The localized failure triggered by the migration of the cart service Dockerfile into a nested subdirectory highlighted the vulnerability of utilizing static, hardcoded directory paths within automation scripts. This empirical evidence proves that CI/CD pipelines must be engineered with the same defensive programming principles applied to application source code. By refactoring the integration logic to incorporate dynamic file-existence validation algorithms, the pipeline successfully adapted to the upstream structural changes. This adaptation demonstrates that resilient, dynamic scripting is a mandatory prerequisite for pipelines designed to handle continuous bulk-merges and decentralized codebase evolution.

The execution of the massive upstream synchronization also illuminated a severe infrastructural vulnerability within the continuous integration server itself. The intense computational and storage overhead required to dynamically build, tag, and orchestrate dozens of microservice containers locally culminated in a fatal resource exhaustion event. The mid-execution crash of the Jenkins

runner node, caused by disk space depletion from orphaned containers and dangling image layers, serves as a critical operational warning. It underscores the absolute necessity of the declarative post-execution cleanup routine, or janitor block. The systematic execution of aggressive container teardowns and forced image pruning is not merely an optional optimization, but a mandatory requirement for maintaining long-term infrastructural stability. CI/CD architectures must be designed to aggressively self-manage their environmental footprints, ensuring that the automation tools do not inadvertently degrade their own hosting infrastructure.

5.6 Security Posture and Multi-Repository Topology

A holistic review of the methodology confirms the strategic soundness of the multi-repository architecture implemented in this study. By strictly segregating the application source code, the GitOps declarative configurations, and the automated testing frameworks into isolated repositories, the system enforces a secure separation of concerns. This decoupling ensures that continuous integration processes, such as code compilation and ephemeral testing, cannot directly or accidentally modify the production state. The Jenkins server is granted only the precise permissions required to perform integration validations and push to the registry, while the Argo CD controller is exclusively responsible for interpreting the validated configuration repository. This isolated topology restricts the blast radius of compromised credentials and prevents unverified application commits from circumventing the mandated quality gates, thereby providing a highly secure and auditable pipeline architecture.

5.7 Technical Limitations and Future Directions

While the architecture successfully fulfilled its design parameters, certain technical limitations and trade-offs must be acknowledged. The End-to-End automated testing suite currently relies on functional, index-based locating strategies and hierarchical child selectors to navigate the Document Object Model. As noted during the integration analysis, this approach is inherently brittle, meaning that even minor, non-functional cosmetic updates to the frontend layout could trigger false-negative test failures, unnecessarily blocking deployments. Additionally, the hardware environment utilized for this study—a localized, single-host virtualized cluster using VirtualBox—inherently lacks the complex networking variability, storage latency, and multi-zone availability challenges present in large-scale cloud provider environments like Google Kubernetes Engine or Amazon EKS. Consequently, while the logical flow of the CI/CD pipeline is validated,

the specific performance metrics and execution speeds observed in this bare-metal simulation may deviate when applied to distributed, globally load-balanced production clusters.

Moving forward, there are several promising avenues for extending and hardening this automated delivery architecture. Future iterations should focus on mitigating testing brittleness by implementing explicit, decoupled test identifiers directly into the frontend source code, thereby creating a more resilient contract between the UI developers and the automation suite. Furthermore, the pipeline's security posture could be significantly enhanced by integrating static application security testing (SAST) and container vulnerability scanning directly into the CI build stages, shifting security analysis as far left as the functional quality gates. From a deployment perspective, the GitOps continuous delivery logic could be upgraded to support advanced release strategies, such as canary deployments or blue-green rollouts, allowing Argo CD to progressively shift live user traffic to newly deployed microservices based on real-time Prometheus monitoring metrics. Ultimately, this study successfully establishes a highly resilient, automated foundation that proves the efficacy of GitOps and ephemeral testing, providing a robust operational baseline capable of supporting continuous, large-scale software evolution.

Appendix

Repositories and uploaded files

- Bulk merge commit: <https://github.com/Stamalexx/microservices-demo-DevOps-apply/commit/dd843275b402baa829ad41206fbd1c04074e8621>
- Initial failure pipeline console output: https://github.com/Stamalexx/microservices-gitops-config/blob/main/jenkins/jenkinsoutput/first_try.txt
- Refactored pipeline code: https://github.com/Stamalexx/microservices-gitops-config/blob/main/jenkins/pipeline_v2/Jenkinsfile
- Final successful merge console output: https://github.com/Stamalexx/microservices-gitops-config/blob/main/jenkins/jenkinsoutput/second_and_final_merge.txt
- GitOps repository, contains value.yaml file and Jenkins pipeline code: <https://github.com/Stamalexx/microservices-gitops-config.git>
- Source code repository: <https://github.com/Stamalexx/microservices-demo-DevOps-apply.git>
- Docker repositories: <https://hub.docker.com/repositories/stamalexx>
- E2E testing repository: <https://github.com/Stamalexx/microservices-playwright-testing.git>

Other Figures

Figure 20 provides a terminal snapshot of the active Kubernetes workloads executing the `kubectl get pods -o wide` command. It serves as empirical confirmation that the entire microservice fleet—including the frontend, backend processors, and stateful datastores—has been successfully provisioned and is in a "Running" state. Furthermore, the extended output details the internal IP address assignments and demonstrates the workload distribution across the worker nodes, confirming that the cluster scheduler is effectively balancing the ephemeral containers across the available virtualized infrastructure.

```
k8s-master@k8s-master:~$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
adsservice-8d9d67f7-zwm7f	1/1	Running	1 (157m ago)	2d20h	10.244.1.44	k8s-worker1	<none>	<none>
cartservice-567f74cb8c-vq744	1/1	Running	1 (157m ago)	2d20h	10.244.2.54	k8s-worker2	<none>	<none>
checkoutservice-5987dfd54c-4fcbbc	1/1	Running	1 (157m ago)	2d20h	10.244.2.56	k8s-worker2	<none>	<none>
currencyservice-57549d95fd-tg4zw	1/1	Running	1 (157m ago)	2d20h	10.244.2.57	k8s-worker2	<none>	<none>
emailservice-7b7898b7f7-gjstw	1/1	Running	2 (149m ago)	2d20h	10.244.1.43	k8s-worker1	<none>	<none>
frontend-8dc76c9c9-8mrlc	1/1	Running	0	87m	10.244.2.71	k8s-worker2	<none>	<none>
frontend-8dc76c9c9-xw4sq	1/1	Running	1 (157m ago)	2d20h	10.244.1.45	k8s-worker1	<none>	<none>
loadgenerator-84dc4f574b-bqsp7	1/1	Running	1 (157m ago)	2d20h	10.244.1.41	k8s-worker1	<none>	<none>
paymentservice-658b997648-rkfl2	1/1	Running	1 (157m ago)	2d20h	10.244.2.61	k8s-worker2	<none>	<none>
productcatalogservice-799b56ff5f-krq88	1/1	Running	1 (157m ago)	2d20h	10.244.1.40	k8s-worker1	<none>	<none>
recommendationservice-6989468ddf-5vzxm	1/1	Running	2 (149m ago)	2d20h	10.244.2.62	k8s-worker2	<none>	<none>
redis-cart-6f8685d6bc-wbhdn	1/1	Running	1 (157m ago)	2d20h	10.244.1.42	k8s-worker1	<none>	<none>
shippingservice-69cbf7c6c7-972kp	1/1	Running	1 (157m ago)	2d20h	10.244.2.58	k8s-worker2	<none>	<none>

Figure 20: Kubernetes Pod Snapshot (*kubectl get pods -o wide*).

Figure 21 and 22 captures the continuous delivery dashboard interface, visualizing the real-time mapping and health status between the active microservices and their corresponding running pods. It illustrates the declarative GitOps synchronization process, confirming that the live cluster state exactly matches the desired state defined within the Git repository. The interface provides visual validation that the deployment pipelines are functioning correctly, with all application components reporting a healthy and fully synced status.

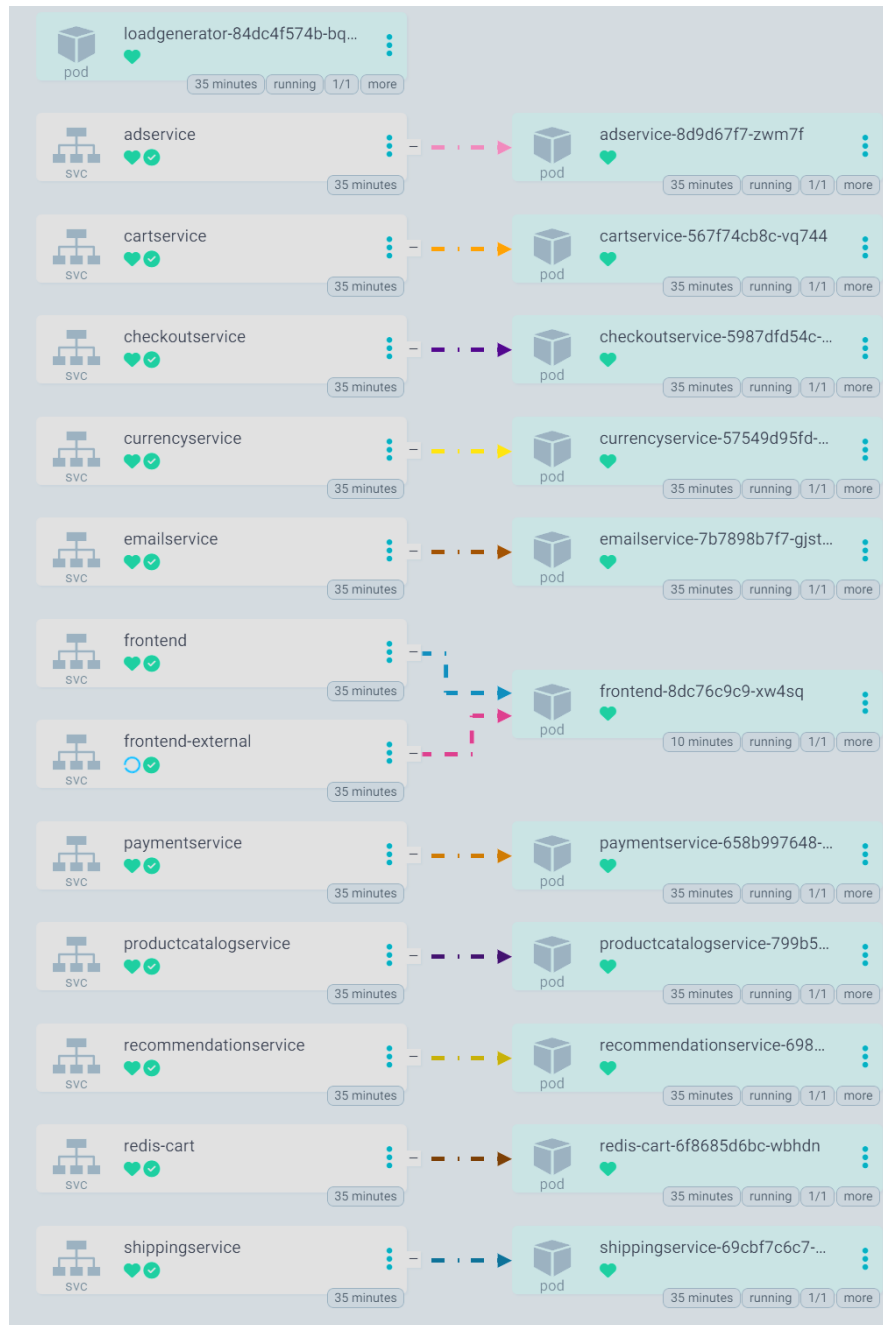


Figure 21: A Kubernetes dashboard visualizing the mapping and health status between active services and their corresponding running pods in a microservices architecture.

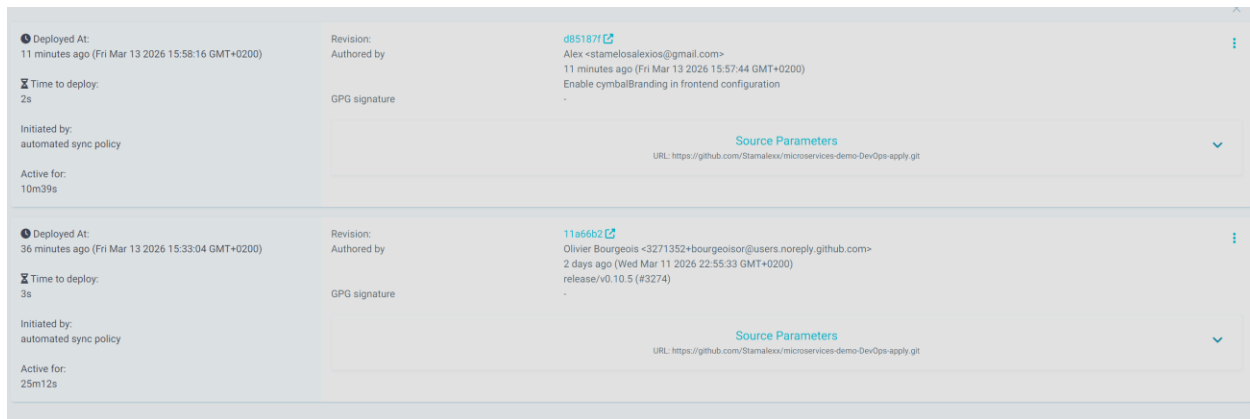


Figure 22: Argo CD snapshot, deployment history. This image shows that the Pipeline is working.

Figure 23 and 24 documents the internal networking configuration of the microservices architecture via the `kubectl get services` command. It displays the dynamically assigned ClusterIPs and dedicated port mappings required for the synchronous gRPC and HTTP communication between the backend services. Additionally, it highlights the ingress routing configuration, showing how external traffic is directed into the cluster to reach the primary frontend application.

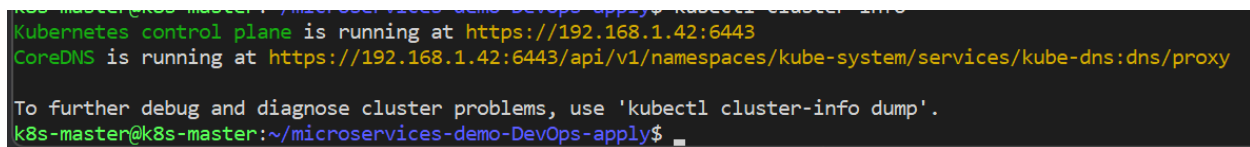


Figure 23: Terminal output of the `kubectl cluster-info` command, confirming the running status and endpoint URLs for the Kubernetes control plane and CoreDNS.

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-master	Ready	control-plane	20h	v1.30.14	192.168.1.42	<none>	Ubuntu 24.04.4 LTS	6.8.0-101-generic	containerd://1.7.28
k8s-worker1	Ready	<none>	174m	v1.30.14	192.168.1.43	<none>	Ubuntu 24.04.4 LTS	6.8.0-101-generic	containerd://1.7.28
k8s-worker2	Ready	<none>	109m	v1.30.14	192.168.1.44	<none>	Ubuntu 24.04.4 LTS	6.8.0-101-generic	containerd://1.7.28

Figure 24: Terminal output showing the detailed status of the Kubernetes cluster nodes (likely from the `kubectl get nodes -o wide` command), displaying one control-plane node and two worker nodes in a 'Ready' state along with their system details.

Figure 25 documents the internal networking configuration of the microservices architecture via the `kubectl get services` command. It displays the dynamically assigned ClusterIPs and dedicated port mappings required for the synchronous gRPC and HTTP communication between the backend services. Additionally, it highlights the ingress routing configuration, showing how external traffic is directed into the cluster to reach the primary frontend application.

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
adservice	ClusterIP	10.104.84.172	<none>	9555/TCP	30m
cartservice	ClusterIP	10.102.242.133	<none>	7070/TCP	30m
checkoutservice	ClusterIP	10.96.3.150	<none>	5050/TCP	30m
currencyservice	ClusterIP	10.107.25.247	<none>	7000/TCP	30m
emailservice	ClusterIP	10.110.250.9	<none>	5000/TCP	30m
frontend	ClusterIP	10.96.244.254	<none>	80/TCP	30m
frontend-external	LoadBalancer	10.108.136.164	<pending>	80:31177/TCP	30m
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	19h
paymentservice	ClusterIP	10.111.91.137	<none>	50051/TCP	30m
productcatalogservice	ClusterIP	10.102.0.248	<none>	3550/TCP	30m
recommendationservice	ClusterIP	10.100.117.243	<none>	8080/TCP	30m
redis-cart	ClusterIP	10.98.124.80	<none>	6379/TCP	30m
shippingservice	ClusterIP	10.104.91.82	<none>	50051/TCP	30m

Figure 25: Terminal output of the `kubectl get services` command, displaying the network configuration, cluster IPs, and assigned ports for the deployed microservices, noting that the `frontend-external` LoadBalancer is currently pending an external IP.

Figure 26 illustrates the foundational hardware virtualization layer provisioned via Oracle VirtualBox, which hosts the local Kubernetes cluster. It visualizes the three-node topology and confirms the strict allocation of computational resources—specifically the base memory (RAM) and processor cores—assigned to each virtual machine. This validates that the local environment possesses sufficient computational capacity to handle both the continuous integration builds and the full-fleet microservice orchestration required for the experimental scenarios.

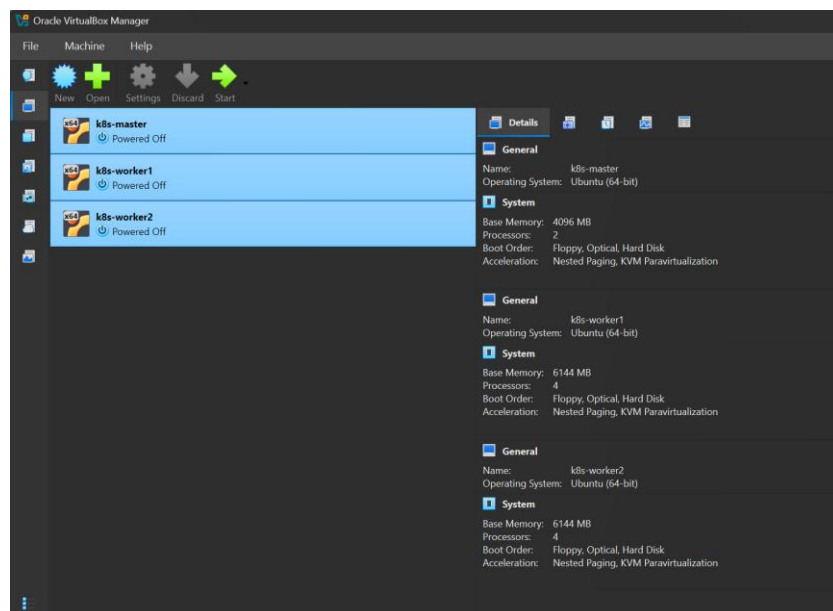


Figure 26: Oracle VirtualBox Manager displaying a three-node Kubernetes cluster setup (one master; two workers), showing the powered-off Ubuntu virtual machines and their allocated system resources like CPU and base memory.

Figure 27 utilizes the `kubectl top nodes` command to display real-time resource utilization metrics across the Kubernetes cluster. By detailing the percentage of CPU and memory consumption for the control-plane and worker nodes, this figure provides insight into the infrastructural overhead required to maintain the microservices architecture. It serves as baseline telemetry data, confirming that the cluster operates within safe resource thresholds during standard deployment states.

```
k8s-master@k8s-master:~$ kubectl top nodes
NAME          CPU(cores)   CPU%   MEMORY(bytes)  MEMORY%
k8s-master    608m        30%    1500Mi         39%
k8s-worker1    184m        4%     1254Mi         21%
k8s-worker2    266m        6%     1316Mi         22%
k8s-master@k8s-master:~$
```

Figure 27: Terminal output of the `kubectl top nodes` command, displaying the current CPU and memory resource utilization across the master and worker nodes in the cluster.

Bibliography

Argo CD (n.d.) OCI - Argo CD - Declarative GitOps CD for Kubernetes. Available at: <https://argo-cd.readthedocs.io/en/latest/user-guide/oci/> (Accessed: 2 June 2026).

Argo CD Documentation (n.d.) *Architectural overview*. Available at: <https://argo-cd.readthedocs.io/en/stable/operator-manual/architecture/> (Accessed: 2 April 2026).

Arora, S., Bhardwaj, A., Kukkar, A. and Kaur, S., 2024, May. A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice-Based Ecosystems. In *2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)* (pp. 621-626). IEEE.

Bass, L., Weber, I. and Zhu, L., 2015. *DevOps: A software architect's perspective*. Addison-Wesley Professional.

Block, A. and Hernandez, C. (2025) *Argo CD: Up and Running: A Hands-On Guide to GitOps and Kubernetes*. Sebastopol, CA: O'Reilly Media. Available at: https://books.google.com/books/about/Argo_CD_Up_and_Running.html?id=eCxmqAAQBAJ (Accessed: 2 April 2026).

Bunnyshell (2026) End-to-End Testing for Microservices: A 2026 Guide. Available at: <https://www.bunnyshell.com/blog/end-to-end-testing-for-microservices-a-2025-guide/> (Accessed: 3 June 2026).

Burns, B., Beda, J., Hightower, K. and Evenson, L., 2022. *Kubernetes: up and running: dive into the future of infrastructure*. " O'Reilly Media, Inc."

Burns, B., Grant, B., Oppenheimer, D., Brewer, E. and Wilkes, J., 2016. Lessons learned from three container-management systems over a decade. *Communications of ACM*.

Carlspring (2024) GitHub - carlspring/idempotence: The idempotence test resource framework. Available at: <https://github.com/carlspring/idempotence> (Accessed: 1 June 2026).

Dev.to (2026) 5 CI/CD Pipeline Disasters I Caused (And How I Fixed Them). Available at: <https://dev.to/sanjaysundarmurthy/5-cicd-pipeline-disasters-i-caused-and-how-i-fixed-them-79j> (Accessed: 4 June 2026).

Docker (2024) Continuous integration with Docker. Available at: <https://docs.docker.com/build/ci/> (Accessed: 4 June 2026).

Docker (2026) Prune networks. Available at: <https://docs.docker.com/engine/manage-resources/pruning/> (Accessed: 4 June 2026).

Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R. and Safina, L., 2017. Microservices: yesterday, today, and tomorrow. *Present and ulterior software engineering*, pp.195-216.

Duvall, P.M., Matyas, S. and Glover, A., 2007. *Continuous integration: improving software quality and reducing risk*. Pearson Education.

Ebert, C., Gallardo, G., Hernantes, J. and Serrano, N., 2016. DevOps. *IEEE software*, 33(3), pp.94-100.

Ephemeral Environments (n.d.) Ephemeral Environments And Testing. Available at: <https://ephemeralenvironments.io/features/testing/> (Accessed: 5 June 2026).

Erich, F.M., Amrit, C. and Daneva, M., 2017. A qualitative study of DevOps usage in practice. *Journal of software: Evolution and Process*, 29(6), p.e1885.

Farid, M., Lim, H.S., Lee, C.P., Zarakovitis, C.C. and Chien, S.F., 2025. Optimizing Kubernetes with Multi-Objective Scheduling Algorithms: A 5G Perspective. *Computers*, 14(9), p.390.

Faustino, D., Gonçalves, N., Portela, M. and Silva, A.R., 2024. Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation. *Performance Evaluation*, 164, p.102411.

Forsgren, N., Humble, J. and Kim, G., 2018. *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution.

GitHub Marketplace (2022) Docker Compose with Tests Action. Available at: <https://github.com/marketplace/actions/docker-compose-with-tests-action> (Accessed: 3 June 2026).

GitLab (2024) Quickly resolve broken CI/CD pipelines with AI. Available at: <https://about.gitlab.com/blog/quickly-resolve-broken-ci-cd-pipelines-with-ai/> (Accessed: 5 June 2026).

Greising, L., Bartel, A. and Hagel, G., 2018, June. Introducing a deployment pipeline for continuous delivery in a software architecture course. In *Proceedings of the 3rd European Conference of Software Engineering Education* (pp. 102-107).

Greising, L., Bartel, A. and Hagel, G., 2018, June. Introducing a deployment pipeline for continuous delivery in a software architecture course. In *Proceedings of the 3rd European Conference of Software Engineering Education* (pp. 102-107).

gRPC (2024) Introduction to gRPC. Available at: <https://grpc.io/docs/what-is-grpc/introduction/> (Accessed: 2 June 2026).

Harbor (n.d.) Creating And Packaging The... Available at: <https://goharbor.io/docs/main/working-with-projects/working-with-oci/working-with-helm-oci-charts/> (Accessed: 2 June 2026).

HashiCorp (n.d.) helm_release | Resources | hashicorp/helm - Terraform Registry. Available at: <https://registry.terraform.io/providers/hashicorp/helm/latest/docs/resources/release> (Accessed: 2 June 2026).

Helm (2019a) Changes Since Helm 2. Available at: https://helm.sh/docs/v3/faq/changes_since_helm2/ (Accessed: 3 June 2026).

Helm (n.d.) *Charts*. Available at: <https://helm.sh/docs/topics/charts/> (Accessed: 1 June 2026).

Humble, J. and Farley, D., 2010. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education.

Hummer, W., et al. (2017) Testing Idempotence for Infrastructure as Code. Available at: <https://dsg.tuwien.ac.at/team/hummer/docs/2013-middleware-iac.pdf> (Accessed: 1 June 2026).

IBM (2023) What is Shift-left Testing? | IBM. Available at: <https://www.ibm.com/think/topics/shift-left-testing> (Accessed: 2 June 2026).

IBM (2023) What is Shift-left Testing? | IBM. Available at: <https://www.ibm.com/think/topics/shift-left-testing> (Accessed: 3 June 2026).

IBM (2025) What is End-to-End (E2E) Testing?. Available at: <https://www.ibm.com/think/topics/end-to-end-testing> (Accessed: 2 June 2026).

Kim, G., Humble, J., Debois, P., Willis, J. and Forsgren, N., 2021. *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution.

Koukis, G., Skaperas, S., Kapetanidou, I.A., Mamatas, L. and Tsaoussidis, V., 2024, June. Performance evaluation of kubernetes networking approaches across constraint edge environments. In *2024 IEEE Symposium on Computers and Communications (ISCC)* (pp. 1-6). IEEE.

Kubernetes (2024) Network Plugins. Available at: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/> (Accessed: 3 June 2026).

Kubernetes (2026a) Container Runtime Interface (CRI). Available at: <https://kubernetes.io/docs/concepts/containers/cri/> (Accessed: 3 June 2026).

Kyverno (2024) Disallow Helm Tiller. Available at: <https://kyverno.io/policies/best-practices/disallow-helm-tiller/disallow-helm-tiller/> (Accessed: 3 June 2026).

Laster, B. (2018) *Jenkins 2: Up and Running: Evolve Your Deployment Pipeline for Next Generation Automation*. Sebastopol, CA: O'Reilly Media.

Liu, L., Xie, D., Cheng, Y. and Li, G., 2021, October. Architecture scheme of DevOps for cross network and multiple environment collaboration. In *Proceedings of the 5th International Conference on Computer Science and Application Engineering* (pp. 1-5).

Luksa, M., 2017. *Kubernetes in action*. Simon and Schuster.

Mazzara, M., Dragoni, N., Bucchiarone, A., Giaretta, A., Larsen, S.T. and Dustdar, S., 2018. Microservices: Migration of a mission critical system. *IEEE Transactions on Services Computing*, 14(5), pp.1464-1477.

Microsoft (2022) Shift testing left with unit tests - Azure DevOps. Available at: <https://learn.microsoft.com/en-us/devops/develop/shift-left-make-testing-fast-reliable> (Accessed: 4 June 2026).

Middleware (2026) Docker cleanup: How to remove Images, containers, and Available at: <https://middleware.io/blog/docker-cleanup/> (Accessed: 6 June 2026).

Momentic (2026) Running Playwright Tests in CI/CD Pipelines: A 101 Guide. Available at: <https://momentic.ai/blog/playwright-ci-cd-guide> (Accessed: 2 June 2026).

Newman, S., 2021. *Building microservices: designing fine-grained systems*. " O'Reilly Media, Inc."

Paavola, E., 2021. Managing Multiple Applications on Kubernetes Using GitOps Principles.

Playwright (n.d.) Auto-waiting. Available at: <https://playwright.dev/docs/actionability> (Accessed: 4 June 2026).

Playwright (n.d.) Playwright: Fast and reliable end-to-end testing for modern web apps. Available at: <https://playwright.dev> (Accessed: 4 June 2026).

Ranger (2025) How Legacy QA Slows CI/CD Pipelines. Available at: <https://www.ranger.net/post/legacy-qa-slows-cicd-pipelines> (Accessed: 4 June 2026).

Rani, V.S., Babu, A.R., Deepthi, K. and Reddy, V.R., 2023, December. Shift-left testing in devops: A study of benefits, challenges, and best practices. In *2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS)* (pp. 1675-1680). IEEE.

Shahin, M., Babar, M.A. and Zhu, L., 2017. Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE access*, 5, pp.3909-3943.

SonarSource (2025) What is Shift Left? Testing, Strategy, Security & Principles. Available at: <https://www.sonarsource.com/resources/library/shift-left/> (Accessed: 2 June 2026).

Soni, M. (2021) *Hands-on Pipeline as YAML with Jenkins*. Delhi: BPB Publications.

SquareOps (2025) CI/CD Pipeline Failures: 7 Common Causes & How to Fix Them Fast. Available at: <https://squareops.com/blog/ci-cd-pipeline-failures-explained-key-debugging-techniques-to-resolve-build-and-deployment-issues/> (Accessed: 4 June 2026).

TestRigor (2025) How to Test in a Microservices Architecture?. Available at: <https://testrigor.com/blog/how-to-test-in-a-microservices-architecture/> (Accessed: 4 June 2026).